

Universidad Politécnica de Chiapas

Carga Inteligente: planificación de rutas y repostaje mediante búsqueda A* en la red vial Tuxtla Gutiérrez – UP Chiapas

Ingeniería en Tecnologías de la Información e Innovación Digital

F U N D A M E N T O S D E I N T E L I G E N C I A A R T I F I C I A L

Presenta:

243790 MAZARIEGOS LAGUNA CARLOS ANDRÉS
243702 CUC LÓPEZ AXEL RODRIGO

Docente

Carlos Alberto Díaz Hernández

Suchiapa, Chiapas, septiembre - diciembre de 2026

Resumen

Este reporte presenta **Carga Inteligente**, un agente que decide **por qué ruta ir, en qué gasolinera cargar y cuántos litros** para viajar en automóvil desde la colonia Chapultepec, en Tuxtla Gutiérrez, hasta la Universidad Politécnica de Chiapas, en Suchiapa, gastando lo menos posible y sin que el tanque llegue a cero. El problema se modeló como una búsqueda en el espacio de estados *(ubicación, litros en el tanque)* sobre un grafo construido con datos reales: tres rutas (Terán, El Jobo y Chiapa de Corzo), diez gasolineras con su precio de gasolina Magna y las distancias de cada tramo.

Se implementaron en Python cuatro estrategias de búsqueda: anchura (BFS), costo uniforme (UCS), avara (Greedy) y A*, esta última con una heurística admisible basada en los litros mínimos que faltan para llegar a la meta y el precio más barato del mapa. El método de búsqueda quedó separado en un módulo propio, y se desarrolló una interfaz gráfica con Tkinter que permite cambiar el inicio, la meta, los litros, el rendimiento, los precios, las distancias y cerrar tramos, además de ejecutar el plan paso a paso.

En los cinco casos presentados, A* encontró siempre el plan de menor costo (igual que UCS) expandiendo entre 1.6 y 3.7 veces menos estados, mientras que BFS y Greedy entregaron planes más caros en los cinco casos. Los resultados muestran que la decisión de dónde y cuánto cargar depende de la ruta, del combustible disponible y de los precios al mismo tiempo, por lo que una regla simple no basta y el problema amerita técnicas de búsqueda informada.

Abstract

This report presents **Carga Inteligente** (*Smart Refueling*), an agent that decides **which route to take, at which gas station to refuel and how many liters to buy** to drive from the Chapultepec neighborhood in Tuxtla Gutiérrez to the Universidad Politécnica de Chiapas in Suchiapa, spending as little money as possible and never running out of fuel. The problem was modeled as a state-space search over *(location, liters in the tank)* on a graph built from real data: three routes (Terán, El Jobo and Chiapa de Corzo), ten gas stations with their regular gasoline price, and the distance of every road segment.

Four search strategies were implemented in Python: breadth-first search (BFS), uniform-cost search (UCS), greedy best-first search and A*, the latter using an admissible heuristic based on the minimum liters still needed to reach the goal and the cheapest price on the map. The search method was separated into its own module, and a Tkinter graphical interface was developed to change the start, the goal, the initial fuel, the fuel efficiency, the prices and distances, to close road segments, and to run the plan step by step.

In the five cases presented, A* always found the minimum-cost plan (as UCS did) while expanding between 1.6 and 3.7 times fewer states, whereas BFS and greedy search returned more expensive plans in all five cases. The results show that deciding where and how much to refuel depends on the route, the available fuel and the prices at the same time, so a simple rule is not enough and the problem calls for informed search techniques.

Índice general

1. Carga Inteligente **8**
- 1.1. Planteamiento del problema . . . 8
- 1.2. PEAS . . . 9
  - 1.2.1. Medida de desempeño . . . 9
  - 1.2.2. Entorno . . . 10
  - 1.2.3. Actuadores . . . 10
  - 1.2.4. Sensores . . . 10
- 1.3. Especificación del problema de búsqueda . . . 11
- 1.4. Estado inicial . . . 12
- 1.5. Meta . . . 12
- 1.6. Espacio de estados . . . 12
- 1.7. Transiciones . . . 13
- 1.8. Métricas de rendimiento . . . 14
- 1.9. Heurísticas . . . 14

2. Implementación **16**
- 2.1. Separación del método de búsqueda . . . 16
- 2.2. Datos del mapa . . . 17
- 2.3. Clase `Escenario` . . . 18
- 2.4. Clase `Problema` . . . 19
  - 2.4.1. Constructor . . . 20
  - 2.4.2. Construcción del grafo . . . 20
  - 2.4.3. Litros mínimos a la meta . . . 21
  - 2.4.4. Prueba de meta . . . 21
  - 2.4.5. Generación de sucesores . . . 22
  - 2.4.6. Heurística . . . 22
- 2.5. Clase `Nodo` . . . 23
- 2.6. Prioridad de cada algoritmo . . . 24
- 2.7. Búsqueda general . . . 24
- 2.8. Funciones auxiliares . . . 26
- 2.9. Interfaz visual . . . 27

3. Resultados **29**
- 3.1. Caso 1: Salida en reserva . . . 29
  - 3.1.1. Descripción . . . 29

3.1.2. Estado inicial y meta . . . 29
3.1.3. Solución . . . 30
3.1.4. Ejecución . . . 32
3.1.5. Comentarios . . . 32
3.2. Caso 2: Obra en el Libramiento Sur . . . 33
3.2.1. Descripción . . . 33
3.2.2. Estado inicial y meta . . . 33
3.2.3. Solución . . . 34
3.2.4. Ejecución . . . 35
3.2.5. Comentarios . . . 36
3.3. Caso 3: Promoción en Gamboa . . . 36
3.3.1. Descripción . . . 36
3.3.2. Estado inicial y meta . . . 36
3.3.3. Solución . . . 37
3.3.4. Ejecución . . . 38
3.3.5. Comentarios . . . 39
3.4. Caso 4: Datos ingresados por el usuario . . . 39
3.4.1. Descripción . . . 39
3.4.2. Estado inicial y meta . . . 39
3.4.3. Solución . . . 40
3.4.4. Ejecución . . . 42
3.4.5. Comentarios . . . 43
3.5. Caso 5: Regreso a casa (cambio de inicio y meta) . . . 44
3.5.1. Descripción . . . 44
3.5.2. Estado inicial y meta . . . 44
3.5.3. Solución . . . 44
3.5.4. Ejecución . . . 46
3.5.5. Comentarios . . . 46
3.6. Comparación general . . . 47

4. Conclusiones y recomendaciones **48**
4.1. Conclusiones . . . 48
4.2. Recomendaciones . . . 49

5. Anexos **50**
5.1. Anexo A: Cómo ejecutar el programa . . . 50
5.2. Anexo B: Caso trivial (tanque suficiente) . . . 50
5.3. Anexo C: Iteraciones completas de A* en el Caso 1 . . . 51
5.4. Anexo D: Repositorio del proyecto . . . 53

Índice de figuras

1.1. Grafo de la red vial Chapultepec – UP Chiapas. Cada caja es una gasolinera con su clave, nombre y precio; cada línea es un tramo con su distancia (elaboración propia con Graphviz). 9

2.1. Diagrama de flujo de la función `buscar()` (elaboración propia con Mermaid). 17
2.2. Interfaz visual: a la izquierda los datos de entrada y a la derecha las salidas. 28

3.1. Caso 1: especificación del estado inicial y la meta en la interfaz. 30
3.2. Caso 1: solución encontrada por A*. 31
3.3. Caso 1: ejecución paso a paso del plan. 32
3.4. Caso 2: especificación con el tramo T3–T4 cerrado. 33
3.5. Caso 2: solución encontrada por A*. 34
3.6. Caso 2: ejecución paso a paso del plan. 35
3.7. Caso 3: especificación con la promoción en J1. 36
3.8. Caso 3: solución encontrada por A*. 37
3.9. Caso 3: ejecución paso a paso del plan. 38
3.10. Caso 4: datos escritos por el usuario en la interfaz. 40
3.11. Caso 4: solución encontrada por A*. 41
3.12. Caso 4: iteraciones de A* mostradas en la interfaz. 42
3.13. Caso 4: ejecución paso a paso del plan. 43
3.14. Caso 5: especificación con inicio en la UP y meta en Chapultepec. 44
3.15. Caso 5: solución encontrada por A*. 45
3.16. Caso 5: ejecución paso a paso del plan. 46
3.17. Estados expandidos por algoritmo en los cinco casos. 47

5.1. Caso trivial: el auto llega sin cargar gasolina. 51

Índice de bloques de código

2.1. Datos del mapa (busqueda.py). 17
2.2. Clase Escenario. 18
2.3. Escenarios predefinidos. 19
2.4. Constructor de la clase Problema. 20
2.5. Construcción del grafo. 20
2.6. Cálculo de los litros mínimos a la meta (Dijkstra). 21
2.7. Prueba de meta. 21
2.8. Generación de sucesores (transiciones). 22
2.9. Función heurística. 22
2.10. Clase Nodo del árbol de búsqueda. 23
2.11. Reconstrucción del plan. 23
2.12. Trayectoria de nodos. 24
2.13. Prioridad de la frontera según el algoritmo. 24
2.14. Búsqueda general (BFS, UCS, Greedy y A*). 24
2.15. Ejecución de los cuatro algoritmos sobre un escenario. 26
2.16. Lugares visitados por el plan. 26
3.1. Plan del Caso 1. 31
3.2. Plan del Caso 2. 34
3.3. Plan del Caso 3. 37
3.4. Plan del Caso 4. 41
3.5. Plan del Caso 5. 45
5.1. Iteraciones de A* en el Caso 1. 51

Índice de tablas

1.1. Gasolineras consideradas y precio de la gasolina Magna. 10
1.2. Caracterización del entorno. 11
1.3. Transiciones de tipo *avanzar* desde cada lugar (rendimiento de 14 km/L). 13
1.4. Transiciones de tipo *cargar* desde el estado (T3, 0.09, falso). 14

2.1. Módulos del proyecto. 16

3.1. Comparación de algoritmos, Caso 1. 31
3.2. Comparación de algoritmos, Caso 2. 35
3.3. Comparación de algoritmos, Caso 3. 38
3.4. Datos de entrada modificados en el Caso 4. 39
3.5. Comparación de algoritmos, Caso 4. 41
3.6. Comparación de algoritmos, Caso 5. 45
3.7. Resumen de los cinco casos resueltos con A*. 47

Capítulo 1

Carga Inteligente

1.1. Planteamiento del problema

Todos los días hacemos el mismo trayecto en automóvil: desde la colonia Chapultepec, en Tuxtla Gutiérrez, hasta la Universidad Politécnica de Chiapas (UP Chiapas), en Suchiapa. El vehículo es un **Chevrolet Aveo 2015** que usa gasolina Magna, con un tanque de **45 L** y un rendimiento que supusimos de **14 km/L**. Es común salir con el tanque casi en reserva, y entonces surgen tres preguntas que dependen unas de otras: **¿por qué ruta ir?, ¿en qué gasolinera cargar? y ¿cuántos litros?**

Para llegar existen tres rutas reales, obtenidas de Google Maps [2]:

- **Ruta Terán**: 24.1 km, aproximadamente 39 minutos.
- **Ruta El Jobo**: 35.4 km, aproximadamente 55 minutos.
- **Ruta Chiapa de Corzo**: 59.1 km, aproximadamente 1 h 23 min.

Las tres rutas se unen en la entrada de Suchiapa y, además, la ruta Terán se conecta con la ruta El Jobo a la altura de la Calzada Emiliano Zapata. Sobre ellas hay diez gasolineras, cada una con un precio distinto de gasolina Magna. La Figura 1.1 muestra el grafo que construimos con esta información y la Tabla 1.1 lista las gasolineras consideradas con su clave y su precio.

El problema **no se puede resolver con una regla simple**, como "ir por la ruta más corta" o "cargar en la gasolinera más barata", porque las tres decisiones están acopladas:

- Una gasolinera barata puede estar en una ruta más larga, y llegar a ella consume litros que también se pagan.
- Con poca gasolina, algunas gasolineras ni siquiera son alcanzables, por lo que el agente está obligado a cargar antes, aunque sea más caro. Por ejemplo, con 0.5 L la gasolinera más barata (Santa Elena, T4) no se alcanza sin cargar primero.
- A veces conviene cargar poco en una gasolinera y completar en otra más adelante.
- Si un tramo se cierra por obra o cambia un precio, la mejor decisión cambia por completo.

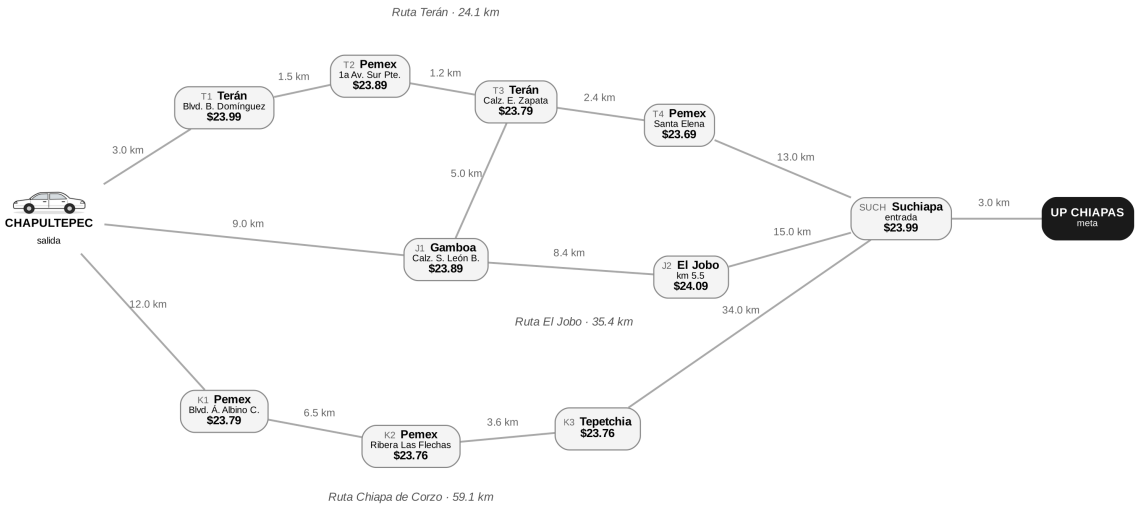


Figura 1.1: Grafo de la red vial Chapultepec – UP Chiapas. Cada caja es una gasolinera con su clave, nombre y precio; cada línea es un tramo con su distancia (elaboración propia con Graphviz).

Por lo tanto hay que explorar combinaciones de (ruta $\times$ gasolinera $\times$ litros), que crecen exponencialmente con el número de intersecciones y estaciones. Esto justifica plantearlo como un **problema de búsqueda en un espacio de estados** [8]: una búsqueda de costo uniforme garantiza encontrar la combinación más barata, y A* la encuentra revisando muchos menos estados gracias a una heurística [3].

1.2. PEAS

El marco PEAS describe a un agente inteligente a partir de su medida de desempeño (*Performance*), su entorno (*Environment*), sus actuadores (*Actuators*) y sus sensores (*Sensors*) [8, 4]. A continuación se describe cada componente para nuestro agente.

1.2.1. Medida de desempeño

Enunciado: minimizar el **gasto total en gasolina ($)** para llegar del inicio a la meta, con la restricción de que **el tanque nunca llegue a cero** en ningún tramo.

Explicación: un plan es mejor que otro si cuesta menos dinero. Un plan en el que el auto se queda sin gasolina no es válido, sin importar su costo. Como medidas secundarias, para comparar algoritmos, se usan los estados expandidos y el tamaño máximo de la frontera (Sección 1.8).

Justificación: el objetivo real del conductor es gastar lo menos posible sin quedarse tirado en la carretera. El tiempo no se incluyó porque las tres rutas son viables y, en el contexto del problema, la preocupación principal es el dinero disponible para la gasolina.

Tabla 1.1: Gasolineras consideradas y precio de la gasolina Magna.

Clave	Gasolinera	Ruta	Precio ($/L)
T1	Terán Blvd. Belisario Domínguez	Terán	23.99
T2	Pemex 1a Av. Sur Poniente	Terán	23.89
T3	Terán Calz. Emiliano Zapata	Terán	23.79
T4	Pemex Santa Elena (Libramiento Sur)	Terán	**23.69**
J1	Gamboa (Calz. Samuel León Brindis)	El Jobo	23.89
J2	El Jobo (km 5.5)	El Jobo	24.09
K1	Pemex Blvd. Ángel Albino Corzo	Chiapa de Corzo	23.79
K2	Pemex Ribera Las Flechas	Chiapa de Corzo	23.76
K3	Tepetchia	Chiapa de Corzo	23.76
SUCH	Entrada de Suchiapa	Común	23.99

1.2.2. Entorno

Enunciado: la red vial Tuxtla Gutiérrez – Suchiapa, representada como un **grafo no dirigido** de 12 nodos y 14 aristas (Figura 1.1).

Explicación: los nodos son el punto de salida (Chapultepec), las diez gasolineras y la UP Chiapas; las aristas son tramos de calle con su distancia en kilómetros, que se convierte en litros consumidos dividiendo entre el rendimiento del auto. Cada gasolinera tiene un precio por litro.

Justificación: el grafo conserva solo lo que importa para decidir: qué lugares se conectan, cuánto combustible se gasta entre ellos y cuánto cuesta cargar en cada uno.

Características del entorno

La Tabla 1.2 clasifica el entorno de tarea y justifica cada característica en términos del problema.

1.2.3. Actuadores

Enunciado: `Desplazarse(vecino)` y `Cargar(`q`)`, con $q \in \{1, 2, 3\}$ litros.

Explicación: en cada intersección el agente elige a qué lugar vecino moverse, y en cada gasolinera puede decidir cargar cierta cantidad de litros. Como salida, el sistema muestra el plan completo y permite abrir la ruta en Google Maps.

Justificación: son las únicas dos decisiones que el conductor toma en el trayecto: por dónde avanzar y cuánto cargar. Se limitó la carga a 1, 2 o 3 litros porque el problema es llegar a la universidad con el gasto mínimo, no llenar el tanque.

1.2.4. Sensores

Enunciado: ubicación actual (GPS), indicador de combustible (litros en el tanque) y precio de la gasolinera en la que se encuentra.

Tabla 1.2: Caracterización del entorno.

Característica	**Justificación**
Determinista	Avanzar por un tramo siempre consume los mismos litros y cargar q litros siempre cuesta $q\times$ precio. No hay resultados aleatorios.
Monoagente	Solo un agente (el conductor) toma decisiones; los demás autos no se modelan.
Completamente observable	El agente conoce su ubicación, los litros en el tanque, el mapa y los precios.
Secuencial	Lo que se carga ahora determina hasta dónde se puede llegar después.
Discreto	Hay un número finito de lugares; las cargas son de 1, 2 o 3 L y los litros se manejan con dos decimales.
Estático	Los precios y el mapa no cambian mientras el agente decide su plan.
Conocido	Se conocen las reglas: consumo por tramo, precio de cada gasolinera y capacidad del tanque.

Explicación: con estos tres datos el agente sabe en qué estado está y qué acciones puede realizar.

Justificación: sin el indicador de combustible el agente no podría saber si alcanza a llegar a la siguiente gasolinera, y sin el precio no podría calcular el costo de cargar.

1.3. Especificación del problema de búsqueda

El problema se especifica formalmente de la siguiente manera:

- **Estado:** la tupla (v, c, b), donde v es la ubicación actual, $c \in [0, 45]$ son los litros en el tanque (con dos decimales) y b es una bandera que indica si el agente ya cargó en esa gasolinera sin haberse movido. La bandera evita que el agente cargue varias veces seguidas en el mismo lugar.
- **Acción avanzar:** $(v_i, c, b) \to (v_j, c - l_{ij}, \text{falso})$, donde $l_{ij} = \frac{d_{ij}}{\text{rendimiento}}$ son los litros que consume el tramo de d_{ij} km. Restricción: $c - l_{ij} \geq 0$. Costo: \$0.
- **Acción cargar:** $(v_i, c, \text{falso}) \to (v_i, c + q, \text{verdadero})$, con $q \in \{1, 2, 3\}$. Restricciones: v_i tiene gasolinera y $c + q \leq 45$. Costo: $q \times p_{v_i}$, donde p_{v_i} es el precio de esa gasolinera.
- **Prueba de meta:** el agente está en la meta, $v = v_{\text{meta}}$ (los litros ya son ≥ 0 por la restricción de avanzar).

- **Costo del camino** $g(n)$**:** la suma del dinero gastado en todas las cargas.
- **Solución:** la secuencia de acciones de menor g que lleva del estado inicial a un estado meta.

1.4. Estado inicial

El estado inicial indica dónde está el auto y con cuántos litros sale; la bandera siempre empieza en falso. Tres ejemplos:

1. $(\text{CHAP}, 0.5, \text{falso})$: sale de Chapultepec casi en reserva. Con 0.5 L alcanza a llegar a Calz. Emiliano Zapata (T3), pero no a Santa Elena (T4), por lo que está obligado a cargar en el camino. Es el estado inicial del Caso 1.
2. $(\text{CHAP}, 5.0, \text{falso})$: sale con 5 L. Como para llegar se necesitan como mínimo 1.72 L, no necesita cargar y el plan óptimo cuesta \$0 (Anexo 5.2).
3. $(\text{UP}, 0.3, \text{falso})$: el auto está en la universidad con 0.3 L y quiere regresar a casa. Con eso solo llega a la gasolinera de Suchiapa con 0.09 L, así que la primera carga es obligatoria. Es el estado inicial del Caso 5.

1.5. Meta

La meta es un lugar del mapa; cualquier cantidad de litros ≥ 0 es válida al llegar. Tres ejemplos:

1. $(\text{UP}, c \geq 0)$: llegar a la universidad. Es la meta de los Casos 1 a 4.
2. $(\text{CHAP}, c \geq 0)$: regresar a casa desde la universidad (Caso 5).
3. $(\text{K2}, c \geq 0)$: llegar a una gasolinera específica, por ejemplo Ribera Las Flechas, para hacer ahí otra actividad. La interfaz permite elegir cualquier nodo como meta.

1.6. Espacio de estados

Cada estado combina un lugar, una cantidad de litros y la bandera. Los litros se manejan con dos decimales entre 0 y 45 L, es decir, hay $45/0.01 + 1 = 4\,501$ valores posibles. La bandera solo puede ser verdadera en los 10 nodos con gasolinera. Por lo tanto, el tamaño máximo del espacio de estados es:

$$|S| = \underbrace{12 \times 4\,501}_{\text{bandera falsa}} + \underbrace{10 \times 4\,501}_{\text{bandera verdadera}} = 54\,012 + 45\,010 = 99\,022 \text{ estados.} \tag{1.1}$$

Para comprobar el cálculo de la Ecuación 1.1, recorrimos con una búsqueda en anchura completa todos los estados alcanzables desde $(\text{CHAP}, 0.5, \text{falso})$ con los datos del Caso 1, y resultaron **97 698 estados alcanzables** (53 688 con la bandera falsa y 44 010 con la

bandera verdadera). En comparación, A* solo expandió 121 de ellos para resolver ese caso (0.12 %).

Los estados más importantes son:

- $(\text{T3}, 0.09, \text{falso})$: el auto llega a Calz. Emiliano Zapata con muy poca gasolina; desde aquí ya no alcanza Santa Elena (necesita 0.17 L), así que debe cargar.
- $(\text{T4}, c, \text{falso})$: el auto está en la gasolinera más barata del mapa; cargar aquí suele ser la mejor opción si se alcanza.
- (SUCH, c, b): la entrada de Suchiapa, por donde pasan las tres rutas; desde aquí solo falta un tramo de 3 km a la UP.

1.7. Transiciones

Desde un estado, las ramas se generan y se analizan en este orden:

1. Primero las acciones **avanzar**, una por cada vecino, en el orden en que aparecen los tramos en la lista `TRAMOS` del código (Listado 2.5).
2. Después las acciones **cargar**, en el orden 1 L, 2 L y 3 L.

Todas las ramas válidas entran a la frontera; la que se analiza primero es la de menor prioridad según el algoritmo, y si dos tienen la misma prioridad se respeta el orden en que se generaron. La Tabla 1.3 lista las transiciones de tipo avanzar, con los litros que consume cada tramo (km ÷ 14 km/L, redondeado a dos decimales).

Tabla 1.3: Transiciones de tipo *avanzar* desde cada lugar (rendimiento de 14 km/L).

Desde	**Hacia**	**Distancia (km)**	**Consumo (L)**
CHAP	T1 / J1 / K1	3.0 / 9.0 / 12.0	0.21 / 0.64 / 0.86
T1	CHAP / T2	3.0 / 1.5	0.21 / 0.11
T2	T1 / T3	1.5 / 1.2	0.11 / 0.09
T3	T2 / T4 / J1	1.2 / 2.4 / 5.0	0.09 / 0.17 / 0.36
T4	T3 / SUCH	2.4 / 13.0	0.17 / 0.93
J1	CHAP / J2 / T3	9.0 / 8.4 / 5.0	0.64 / 0.60 / 0.36
J2	J1 / SUCH	8.4 / 15.0	0.60 / 1.07
K1	CHAP / K2	12.0 / 6.5	0.86 / 0.46
K2	K1 / K3	6.5 / 3.6	0.46 / 0.26
K3	K2 / SUCH	3.6 / 34.0	0.26 / 2.43
SUCH	T4 / J2 / K3 / UP	13.0 / 15.0 / 34.0 / 3.0	0.93 / 1.07 / 2.43 / 0.21
UP	SUCH	3.0	0.21

Por ejemplo, desde el estado inicial $(\text{CHAP}, 0.5, \text{falso})$ se generan tres posibles ramas de avanzar: hacia T1 (consume 0.21 L), hacia J1 (consume 0.64 L) y hacia K1 (consume

0.86 L). Solo la primera es válida, porque en las otras dos el tanque quedaría en negativo; así, la única transición es $(\text{T1}, 0.29, \text{falso})$, y como en Chapultepec no hay gasolinera, no hay acciones de carga. La Tabla 1.4 muestra las transiciones de carga desde un estado en la gasolinera T3.

Tabla 1.4: Transiciones de tipo *cargar* desde el estado (T3, 0.09, falso).

Acción	Estado resultante	Costo
Cargar 1 L	(T3, 1.09, verdadero)	$23.79
Cargar 2 L	(T3, 2.09, verdadero)	$47.58
Cargar 3 L	(T3, 3.09, verdadero)	$71.37

1.8. Métricas de rendimiento

Para evaluar y comparar los algoritmos se seleccionaron las siguientes métricas:

- **Gasto total ($):** el costo g del plan encontrado. Es la métrica principal porque es lo que el agente quiere minimizar.
- **¿Óptimo?:** si el gasto es igual al de UCS, que siempre encuentra el plan más barato. Permite saber si un algoritmo es confiable.
- **Estados expandidos:** cuántos estados se sacaron de la frontera y se revisaron. Mide el esfuerzo de búsqueda; entre dos algoritmos óptimos, es mejor el que expande menos.
- **Frontera máxima:** el mayor número de nodos que hubo en la frontera al mismo tiempo. Mide el uso de memoria.
- **Tiempo (ms):** tiempo de ejecución medido con `time.perf_counter()`. Es complementario, porque depende de la computadora.
- **Litros al llegar:** con cuántos litros llega el auto a la meta; sirve para verificar que el plan es válido.

1.9. Heurísticas

La heurística estima, de forma optimista, cuánto dinero falta gastar desde un estado:

$$h(v, c) = \max(0,\ L_{\text{mín}}(v) - c) \times p_{\text{mín}} \tag{1.2}$$

donde $L_{\text{mín}}(v)$ son los litros mínimos para llegar de v a la meta por la ruta de menor consumo (calculados con el algoritmo de Dijkstra [1] desde la meta) y $p_{\text{mín}}$ es el precio más barato del mapa. Si el tanque ya trae suficiente gasolina, $h = 0$.

Justificación (admisibilidad): la heurística de la Ecuación 1.2 nunca sobreestima el costo real, por dos razones:

1. Para llegar a la meta hay que recorrer al menos $L_{\text{mín}}(v)$ litros; si el tanque trae c, hay que comprar al menos $L_{\text{mín}}(v) - c$ litros.

2. Esos litros no pueden costar menos que $p_{\text{mín}}$, el precio más barato del mapa.

Por lo tanto $h(n) \leq h^*(n)$ para todo estado, y A* con esta heurística es **óptimo** [3, 8, 9]. Si un lugar ya no tiene camino a la meta (por ejemplo, por tramos cerrados), $h = \infty$ y la rama se descarta.

Ejemplo: en el estado inicial del Caso 1, $h(\text{CHAP}, 0.5) = (1.72 - 0.5) \times 23.69 = \$28.90$, y el costo real del plan óptimo es \$47.48, por lo que h no sobreestima.

Heurísticas alternativas consideradas:

- $h = 0$: es admisible, pero convierte a A* en UCS y no reduce los estados expandidos.
- Distancia en línea recta $\times$ consumo $\times$ precio: la descartamos porque no toma en cuenta la gasolina que ya trae el tanque y puede sobreestimar el costo cuando el auto ya tiene suficiente combustible.

Capítulo 2

Implementación

El sistema se programó en **Python 3**. Para la frontera ordenada se usó el módulo `heapq` [6], para definir los escenarios y los nodos del árbol de búsqueda se usó `dataclasses` [5], y la interfaz gráfica se hizo con `tkinter` [7], que ya viene incluido con Python. Las instrucciones para ejecutar el programa están en el Anexo 5.1 y el repositorio con el código completo se describe en el Anexo 5.4.

2.1. Separación del método de búsqueda

Como se pidió, el **método de búsqueda está separado** de la visualización. El proyecto tiene tres archivos (Tabla 2.1): `busqueda.py` contiene únicamente el modelo del problema y los algoritmos, y no imprime ni dibuja nada; los otros dos archivos solo lo importan y muestran sus resultados.

Tabla 2.1: Módulos del proyecto.

Archivo	**Contenido**
`busqueda.py`	**Método de búsqueda**: datos del mapa, escenarios, clase `Problema` (estados, sucesores, meta y heurística), clase `Nodo` y los algoritmos BFS, UCS, Greedy y A*.
`interfaz.py`	**Visualización**: ventana de Tkinter para capturar los datos de entrada y mostrar el mapa, el plan, la ejecución paso a paso, la comparación y las iteraciones.
`carga_inteligente.py`	**Visualización en consola**: menú de escenarios, tablas en texto, gráfica con matplotlib y enlace de Google Maps.

Los cuatro algoritmos comparten una misma función de búsqueda; lo único que cambia entre ellos es la prioridad con la que se ordena la frontera. La Figura 2.1 muestra el diagrama de flujo de esa función. En las siguientes secciones se documenta, con un bloque de código, cada función y clase de `busqueda.py`, indicando qué parámetros recibe, qué hace y qué devuelve.

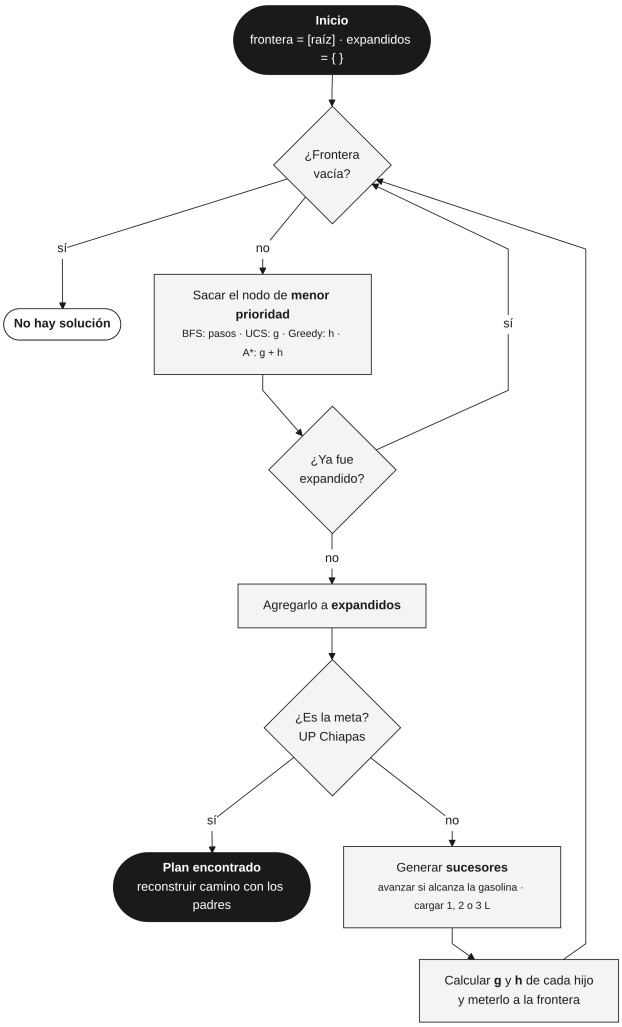


Figura 2.1: Diagrama de flujo de la función `buscar()` (elaboración propia con Mermaid).

2.2. Datos del mapa

El Listado 2.1 contiene las constantes del problema: rendimiento, capacidad del tanque, cargas posibles, nombre de cada nodo, precio de cada gasolinera y la lista de tramos con su distancia. Estos datos son los valores por defecto; cualquier escenario puede cambiarlos.

```
RENDIMIENTO = 14.0              # km por litro (Chevrolet Aveo 2015, supuesto)
CAPACIDAD = 45.0                # litros que le caben al tanque
CARGAS_POSIBLES = (1, 2, 3)     # litros que se pueden cargar en cada parada
ALGORITMOS = ("BFS", "UCS", "Greedy", "A*")

# Nombre completo de cada nodo del grafo
NOMBRES = {
    "CHAP": "Chapultepec (casa)",
    "T1": "Gasolinera Terán Blvd. Belisario Domínguez",
```

```
    "T2": "Pemex 1a Av. Sur Poniente",
    "T3": "Gasolinera Terán Calz. Emiliano Zapata",
    "T4": "Pemex Santa Elena (Libramiento Sur Pte.)",
    "J1": "Gasolinera Gamboa (Calz. Samuel León Brindis)",
    "J2": "Gasolinera El Jobo (km 5.5)",
    "K1": "Pemex Blvd. Ángel Albino Corzo",
    "K2": "Pemex Ribera Las Flechas",
    "K3": "Gasolinera Tepetchia",
    "SUCH": "Gasolinera entrada de Suchiapa",
    "UP": "UP Chiapas",
}

# Precio normal de la Magna ($/L). Los nodos que no están aquí no tienen gasolinera.
PRECIOS_BASE = {
    "T1": 23.99, "T2": 23.89, "T3": 23.79, "T4": 23.69,
    "J1": 23.89, "J2": 24.09,
    "K1": 23.79, "K2": 23.76, "K3": 23.76,
    "SUCH": 23.99,
}

# Tramos de calle (A, B, km). Se puede circular en ambos sentidos.
TRAMOS = [
    ("CHAP", "T1", 3.0), ("T1", "T2", 1.5), ("T2", "T3", 1.2),
        # Ruta Terán
    ("T3", "T4", 2.4), ("T4", "SUCH", 13.0),
    ("CHAP", "J1", 9.0), ("J1", "J2", 8.4), ("J2", "SUCH", 15.0),
        # Ruta El Jobo
    ("CHAP", "K1", 12.0), ("K1", "K2", 6.5), ("K2", "K3", 3.6),
        # Ruta Chiapa
    ("K3", "SUCH", 34.0),
    ("T3", "J1", 5.0),
        # Conexión Terán--Jobo
    ("SUCH", "UP", 3.0),
        # Tramo final común
]
```

Listing 2.1: Datos del mapa (busqueda.py).

2.3. Clase `Escenario`

```
@dataclass
class Escenario:
    nombre: str
    descripcion: str
    litros_iniciales: float
    precios_especiales: dict = field(default_factory=dict)   # {nodo: precio nuevo}
    tramos_cerrados: list = field(default_factory=list)      # [(A, B), ...]
    km_especiales: dict = field(default_factory=dict)        # {(A, B): km nuevos}
```

```
    rendimiento: float = RENDIMIENTO                    # km/L del auto
    capacidad: float = CAPACIDAD                        # litros del tanque
    inicio: str = "CHAP"                                # de dónde sale
    meta: str = "UP"                                    # a dónde quiere llegar
```

Listing 2.2: Clase Escenario.

- **Parámetros:** `nombre` y `descripcion` (texto); `litros_iniciales` (número); opcionalmente `precios_especiales` (diccionario nodo → precio), `tramos_cerrados` (lista de pares de nodos), `km_especiales` (diccionario tramo → km), `rendimiento`, `capacidad`, `inicio` y `meta`.
- **Qué hace:** guarda todos los datos que pueden cambiar de un caso a otro. Los campos que no se indican toman los valores por defecto del Listado 2.1. La interfaz gráfica construye un objeto de esta clase con lo que escribe el usuario.
- **Devuelve:** un objeto `Escenario`. Los cuatro escenarios predefinidos se muestran en el Listado 2.3.

```
ESCENARIOS = [
    Escenario("1. En reserva",
              "Sale con 0.5 L, casi vacío. Tiene que cargar en el camino.",
              0.5),
    Escenario("2. Tanque suficiente",
              "Sale con 5 L. Le alcanza para todo el viaje.",
              5.0),
    Escenario("3. Obra en el Libramiento Sur",
              "Sale con 0.5 L y el tramo Calz. Emiliano Zapata -- Santa Elena está cerrado.",
              0.5, tramos_cerrados=[("T3", "T4")]),
    Escenario("4. Promoción en El Jobo",
              "Sale con 0.5 L y la Gasolinera Gamboa tiene la Magna a $21.49.",
              0.5, precios_especiales={"J1": 21.49}),
]
```

Listing 2.3: Escenarios predefinidos.

2.4. Clase `Problema`

Esta clase define el problema de búsqueda de la Sección 1.3: construye el grafo, calcula los datos de la heurística y genera los sucesores.

2.4.1. Constructor

```
def __init__(self, escenario):
    """Prepara todo lo que necesita la búsqueda a partir de un
   Escenario."""
    self.escenario = escenario
    self.precios = {**PRECIOS_BASE, **escenario.precios_especiales}
    self.rendimiento = escenario.rendimiento
    self.capacidad = escenario.capacidad
    self.meta = escenario.meta
    self.km = {frozenset(k): v for k, v in escenario.km_especiales.
   items()}
    self.grafo = self._construir_grafo(escenario.tramos_cerrados)
    self.litros_minimos = self._calcular_litros_minimos()
    self.precio_minimo = min(self.precios.values())
    self.inicio = (escenario.inicio, escenario.litros_iniciales,
   False)
```

Listing 2.4: Constructor de la clase Problema.

- **Parámetros:** `escenario`: un objeto `Escenario` (Listado 2.2).
- **Qué hace:** combina los precios por defecto con los del escenario, guarda el rendimiento, la capacidad y la meta, construye el grafo (Listado 2.5), calcula los litros mínimos a la meta (Listado 2.6), obtiene el precio más barato y arma el estado inicial (v_0, c_0, falso).
- **Devuelve:** nada; deja listo el objeto para usarlo en la búsqueda.

2.4.2. Construcción del grafo

```
def _construir_grafo(self, cerrados):
    """Lista de tramos -> diccionario {lugar: {vecino: litros}}."""
    cerrados = {frozenset(t) for t in cerrados}
    grafo = {n: {} for n in NOMBRES}
    for a, b, km in TRAMOS:
        if frozenset((a, b)) in cerrados:          # tramo cerrado:
   no se agrega
            continue
        km = self.km.get(frozenset((a, b)), km)    # km editados (si
   los hay)
        litros = round(km / self.rendimiento, 2)  # km ÷ rendimiento
    = litros
        grafo[a][b] = litros
        grafo[b][a] = litros
    return grafo
```

Listing 2.5: Construcción del grafo.

- **Parámetros:** `cerrados`: lista de tramos cerrados, cada uno como par de nodos.

- **Qué hace:** recorre la lista `TRAMOS`, omite los tramos cerrados, reemplaza los km si el escenario los cambió y convierte cada distancia a litros (km $\div$ rendimiento). Guarda cada tramo en ambos sentidos porque el grafo es no dirigido.
- **Devuelve:** un diccionario {`lugar: {vecino: litros}`}.

2.4.3. Litros mínimos a la meta

```
def _calcular_litros_minimos(self):
    """Dijkstra desde la META: litros mínimos para llegar desde cada
    lugar."""
    minimos, frontera = {}, [(0.0, self.meta)]
    while frontera:
        litros, lugar = heapq.heappop(frontera)
        if lugar in minimos:
            continue
        minimos[lugar] = round(litros, 2)
        for vecino, gasto in self.grafo[lugar].items():
            heapq.heappush(frontera, (litros + gasto, vecino))
    return minimos
```

Listing 2.6: Cálculo de los litros mínimos a la meta (Dijkstra).

- **Parámetros:** ninguno (usa `self.grafo` y `self.meta`).
- **Qué hace:** aplica el algoritmo de Dijkstra [1] partiendo desde la meta: saca siempre el lugar con menos litros acumulados y propaga a sus vecinos. Así obtiene, para cada lugar, el consumo de la ruta más corta hasta la meta. Es el valor $L_{\text{mín}}(v)$ de la heurística (Ecuación 1.2).
- **Devuelve:** un diccionario {`lugar: litros mínimos`}. Los lugares sin camino a la meta no aparecen.

2.4.4. Prueba de meta

```
def es_meta(self, estado):
    """True si el auto está en el lugar meta (los litros ya son >=
    0)."""
    return estado[0] == self.meta
```

Listing 2.7: Prueba de meta.

- **Parámetros:** `estado`: tupla `(lugar, litros, ya_cargo)`.
- **Qué hace:** compara el lugar del estado con la meta. No hace falta revisar los litros porque la acción avanzar nunca deja el tanque en negativo.
- **Devuelve:** `True` si el estado es meta, `False` en otro caso.

2.4.5. Generación de sucesores

```
def sucesores(self, estado):
    """Genera (acción, nuevo_estado, costo) para cada acción válida.
    """
    lugar, litros, ya_cargo = estado

    # Acción AVANZAR: cuesta $0, pero gasta litros. Solo si alcanza
    la gasolina.
    for vecino, gasto in self.grafo[lugar].items():
        restante = round(litros - gasto, 2)
        if restante >= 0:
            yield f"Ir a {NOMBRES[vecino]}", (vecino, restante,
    False), 0.0

    # Acción CARGAR: solo en gasolinera, una vez por parada y sin
    rebasar el tanque.
    if lugar in self.precios and not ya_cargo:
        for q in CARGAS_POSIBLES:
            if litros + q <= self.capacidad:
                costo = round(q * self.precios[lugar], 2)
                yield (f"Cargar {q} L en {NOMBRES[lugar]} (${costo
    :.2f})",
                       (lugar, round(litros + q, 2), True), costo)
```

Listing 2.8: Generación de sucesores (transiciones).

- **Parámetros:** estado: tupla `(lugar, litros, ya_cargo)`.
- **Qué hace:** genera las transiciones válidas en el orden descrito en la Sección 1.7: primero avanzar a cada vecino al que le alcance la gasolina (costo $0, bandera en falso) y después, si hay gasolinera y no ha cargado ahí, cargar 1, 2 o 3 litros sin rebasar el tanque (costo $q\times$ precio, bandera en verdadero). Usa `yield` para entregar las transiciones una por una.
- **Devuelve:** un generador de tuplas `(acción, nuevo_estado, costo)`; la acción es un texto descriptivo para mostrar el plan.

2.4.6. Heurística

```
def heuristica(self, estado):
    """h = litros que faltan × precio más barato. Nunca exagera (
    ADMISIBLE)."""
    lugar, litros, _ = estado
    if lugar not in self.litros_minimos:          # desde aquí ya no
    se llega a la meta
        return float("inf")
    faltan = max(0.0, self.litros_minimos[lugar] - litros)
    return round(faltan * self.precio_minimo, 2)
```

Listing 2.9: Función heurística.

- **Parámetros:** `estado`: tupla `(lugar, litros, ya_cargo)`.
- **Qué hace:** calcula la Ecuación 1.2: los litros que faltan comprar multiplicados por el precio más barato. Si el lugar ya no tiene camino a la meta devuelve infinito para descartar esa rama.
- **Devuelve:** un número ≥ 0 (o infinito) con el costo estimado en pesos.

2.5. Clase `Nodo`

```python
@dataclass
class Nodo:
    estado: tuple
    g: float = 0.0                  # pesos gastados desde el inicio
    pasos: int = 0                  # acciones realizadas
    padre: "Nodo" = None            # nodo del que venimos
    accion: str = ""                # acción que nos trajo aquí
```

Listing 2.10: Clase Nodo del árbol de búsqueda.

- **Parámetros:** `estado` (tupla), `g` (dinero gastado desde el inicio), `pasos` (acciones realizadas), `padre` (nodo anterior) y `accion` (texto de la acción que llevó a este nodo).
- **Qué hace:** representa un nodo del árbol de búsqueda. Guardar al padre permite reconstruir el plan sin copiar la lista de acciones en cada nodo.
- **Devuelve:** un objeto `Nodo`.

```python
def camino(self):
    """Reconstruye la lista de acciones siguiendo a los padres."""
    acciones, nodo = [], self
    while nodo.padre is not None:
        acciones.append(nodo.accion)
        nodo = nodo.padre
    return acciones[::-1]
```

Listing 2.11: Reconstrucción del plan.

- **Parámetros:** ninguno (usa el nodo actual).
- **Qué hace:** sube de padre en padre hasta la raíz guardando la acción de cada nodo, y al final invierte la lista.
- **Devuelve:** la lista de acciones en orden, desde el inicio hasta este nodo.

```python
def trayectoria(self):
    """Lista de nodos desde la raíz hasta este (para ejecutar paso a paso)."""
    nodos, nodo = [], self
    while nodo is not None:
        nodos.append(nodo)
        nodo = nodo.padre
    return nodos[::-1]
```

Listing 2.12: Trayectoria de nodos.

- **Parámetros:** ninguno.
- **Qué hace:** igual que `camino()`, pero guarda los nodos completos (estado, gasto y acción). La interfaz la usa para ejecutar el plan paso a paso.
- **Devuelve:** la lista de nodos desde la raíz hasta este nodo.

2.6. Prioridad de cada algoritmo

```python
def prioridad(algoritmo, nodo, h):
    """Número con el que se ordena la frontera según el algoritmo."""
    return {"BFS": nodo.pasos, "UCS": nodo.g, "Greedy": h}.get(algoritmo, nodo.g + h)
```

Listing 2.13: Prioridad de la frontera según el algoritmo.

- **Parámetros:** `algoritmo` (`"BFS"`, ÜCS", `"Greedy"` o .ª*"), `nodo` (objeto `Nodo`) y `h` (valor heurístico del nodo).
- **Qué hace:** elige con qué número se ordena la frontera: BFS usa el número de pasos (primero lo más superficial), UCS usa g (primero lo más barato), Greedy usa h (primero lo que parece más cercano a la meta) y A* usa $f = g + h$.
- **Devuelve:** el número de prioridad; la frontera siempre saca el menor.

2.7. Búsqueda general

```python
def buscar(problema, algoritmo, registro=None):
    """Búsqueda general con frontera ordenada y cycle checking.
    registro: lista opcional donde se guarda cada iteración (texto).
    """
    raiz = Nodo(problema.inicio)
    frontera = [(0, 0, raiz)]          # (prioridad, desempate, nodo)
    expandidos = set()                 # cycle checking
    contador, frontera_max = 0, 1
```

```
    t0 = time.perf_counter()

    while frontera:
        _, _, nodo = heapq.heappop(frontera)          # 1) sacar el de menor prioridad
        if nodo.estado in expandidos:
            continue
        expandidos.add(nodo.estado)

        if registro is not None:
            h = problema.heuristica(nodo.estado)
            lugar, litros, _ = nodo.estado
            registro.append(f"It. {len(expandidos):3}: ({lugar:>4}, {litros:5.2f} L)"
                            f"  g = {nodo.g:6.2f}  h = {h:6.2f}  f = {nodo.g + h:6.2f}")

        if problema.es_meta(nodo.estado):             # 2) ¿es la meta? (al SACARLA)
            return {"costo": round(nodo.g, 2), "camino": nodo.camino(),
                    "expandidos": len(expandidos), "frontera_max": frontera_max,
                    "tiempo_ms": (time.perf_counter() - t0) * 1000,
                    "litros_finales": nodo.estado[1], "nodo": nodo}

        for accion, nuevo, costo in problema.sucesores(nodo.estado):   # 3) expandir
            if nuevo in expandidos:
                continue
            hijo = Nodo(nuevo, round(nodo.g + costo, 2), nodo.pasos + 1, nodo, accion)
            h = problema.heuristica(nuevo)
            if h == float("inf"):                     # callejón sin salida: se descarta
                continue
            contador += 1
            heapq.heappush(frontera, (prioridad(algoritmo, hijo, h), contador, hijo))
        frontera_max = max(frontera_max, len(frontera))

    return None                                       # frontera vacía: no hay solución
```

Listing 2.14: Búsqueda general (BFS, UCS, Greedy y A*).

- **Parámetros:** `problema` (objeto `Problema`), `algoritmo` (texto) y `registro` (lista opcional donde se guarda el texto de cada iteración).
- **Qué hace:** implementa el diagrama de la Figura 2.1. La frontera es un *heap* con tuplas `(prioridad, desempate, nodo)`; el contador de desempate respeta el orden de generación cuando dos prioridades son iguales. En cada iteración saca el nodo de menor prioridad, lo descarta si ya fue expandido (*cycle checking*), revisa si

es meta y, si no, genera sus hijos con `sucesores()`, calcula su prioridad y los mete a la frontera. **Solo termina cuando la meta se saca de la frontera**, no cuando se genera, lo que garantiza la optimalidad de UCS y A*.

- **Devuelve:** un diccionario con el costo, la lista de acciones, los estados expandidos, la frontera máxima, el tiempo en milisegundos, los litros al llegar y el nodo final; o `None` si la frontera se vacía (no hay solución).

2.8. Funciones auxiliares

```
def resolver(escenario, registro=None):
    """Corre los 4 algoritmos sobre un escenario. Devuelve (problema
    , resultados).
    Las iteraciones que se guardan en 'registro' son las de A*."""
    problema = Problema(escenario)
    resultados = {alg: buscar(problema, alg, registro if alg == "A*"
    else None)
                  for alg in ALGORITMOS}
    return problema, resultados
```

Listing 2.15: Ejecución de los cuatro algoritmos sobre un escenario.

- **Parámetros:** `escenario` (objeto `Escenario`) y `registro` (lista opcional para las iteraciones de A*).
- **Qué hace:** crea el `Problema` y ejecuta `buscar()` con los cuatro algoritmos. Es la única función que llaman la interfaz y la consola.
- **Devuelve:** una tupla `(problema, resultados)`, donde `resultados` es un diccionario `{algoritmo: resultado}`.

```
def lugares_del_plan(nodo):
    """Lista de lugares por los que pasa el plan (sin repetir
    paradas)."""
    lugares = []
    for n in nodo.trayectoria():
        if not lugares or lugares[-1] != n.estado[0]:
            lugares.append(n.estado[0])
    return lugares
```

Listing 2.16: Lugares visitados por el plan.

- **Parámetros:** `nodo`: el nodo meta encontrado por la búsqueda.
- **Qué hace:** recorre la trayectoria y guarda cada lugar sin repetirlo cuando el auto se queda en el mismo sitio para cargar.
- **Devuelve:** la lista de lugares de la ruta, por ejemplo `[CHAP, T1, T2, T3, T4, SUCH, UP]`.

2.9. Interfaz visual

La interfaz (`interfaz.py`) solo captura datos y muestra resultados; toda la búsqueda la hace llamando a `resolver()` (Listado 2.15). La Figura 2.2 muestra la ventana. Desde ella se puede:

- **Cambiar el escenario**: elegir uno de los cuatro escenarios predefinidos y cargar sus datos.
- **Cambiar el inicio y la meta**: elegir cualquiera de los 12 lugares del mapa en las listas *Inicio* y *Meta*.
- **Cambiar los datos del vehículo**: litros iniciales, rendimiento (km/L) y capacidad del tanque.
- **Cambiar el mapa**: el precio de cada gasolinera, los kilómetros de cada tramo y marcar tramos como cerrados.
- **Ver las salidas**: el mapa con la ruta, el resumen (gasto, litros al llegar, h del inicio y ruta), la pestaña con el plan, la comparación de los cuatro algoritmos, las iteraciones de A* y un botón para abrir la ruta en Google Maps.
- **Ejecutar el plan paso a paso**: con los botones *Inicio*, *Anterior*, *Siguiente*, *Final* y *Reproducir*, el Aveo se mueve sobre el mapa mostrando la acción, los litros en el tanque y el dinero gastado en cada paso. Los tramos ya recorridos se dibujan en negro grueso y los que faltan en línea punteada.

Antes de buscar, la interfaz valida que los datos sean números, que no sean negativos, que los litros iniciales no superen la capacidad del tanque y que el inicio sea distinto de la meta. Si algo está mal muestra un mensaje en lugar de cerrarse. Si con los datos no existe forma de llegar, el resumen indica que no hay solución.

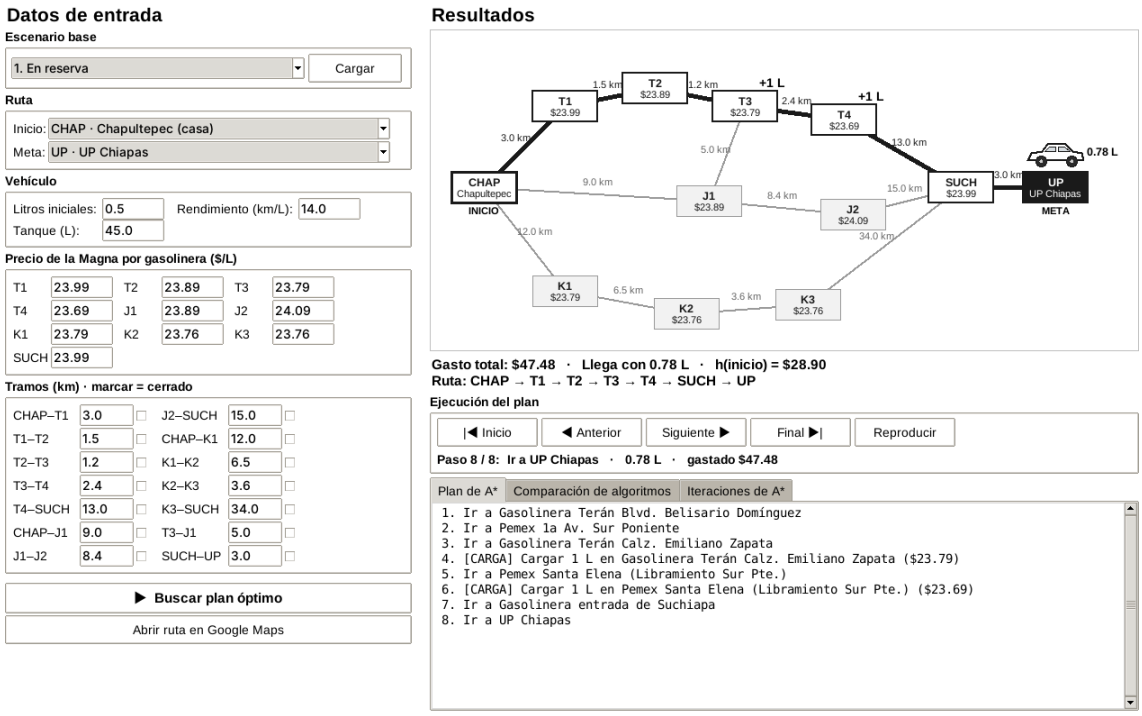


Figura 2.2: Interfaz visual: a la izquierda los datos de entrada y a la derecha las salidas.

Capítulo 3

Resultados

En este capítulo se presentan cinco casos no triviales resueltos con la interfaz visual (Sección 2.9). Para cada uno se describe la situación, se muestra cómo se especificaron el estado inicial y la meta, la solución encontrada por A*, la comparación con los otros tres algoritmos y la ejecución del plan paso a paso. El caso trivial, en el que el auto no necesita cargar, se incluye en el Anexo 5.2. Al final, la Sección 3.6 compara los cinco casos.

En todas las capturas, la ruta de A* se dibuja en **negro grueso**; durante la ejecución, los tramos que faltan por recorrer aparecen en línea punteada, el Aveo blanco indica la posición del auto con los litros que trae, y junto a cada gasolinera se indica cuánto se cargó (por ejemplo, **+1 L**).

3.1. Caso 1: Salida en reserva

3.1.1. Descripción

El auto sale de Chapultepec con **0.5 L**, casi vacío, rumbo a la UP Chiapas, con los precios y distancias normales. Es el caso base: el agente debe decidir dónde cargar sabiendo que la gasolinera más barata (Santa Elena, T4) no es alcanzable con la gasolina que trae.

3.1.2. Estado inicial y meta

Estado inicial: $(\text{CHAP}, 0.5, \text{falso})$. Meta: $(\text{UP}, c \geq 0)$. En la interfaz se eligió el escenario base *1. En reserva*, que llena todos los campos (Figura 3.1). Los litros mínimos para llegar son 1.72 L, por lo que $h(\text{inicio}) = (1.72 - 0.5) \times 23.69 = \$28.90$.

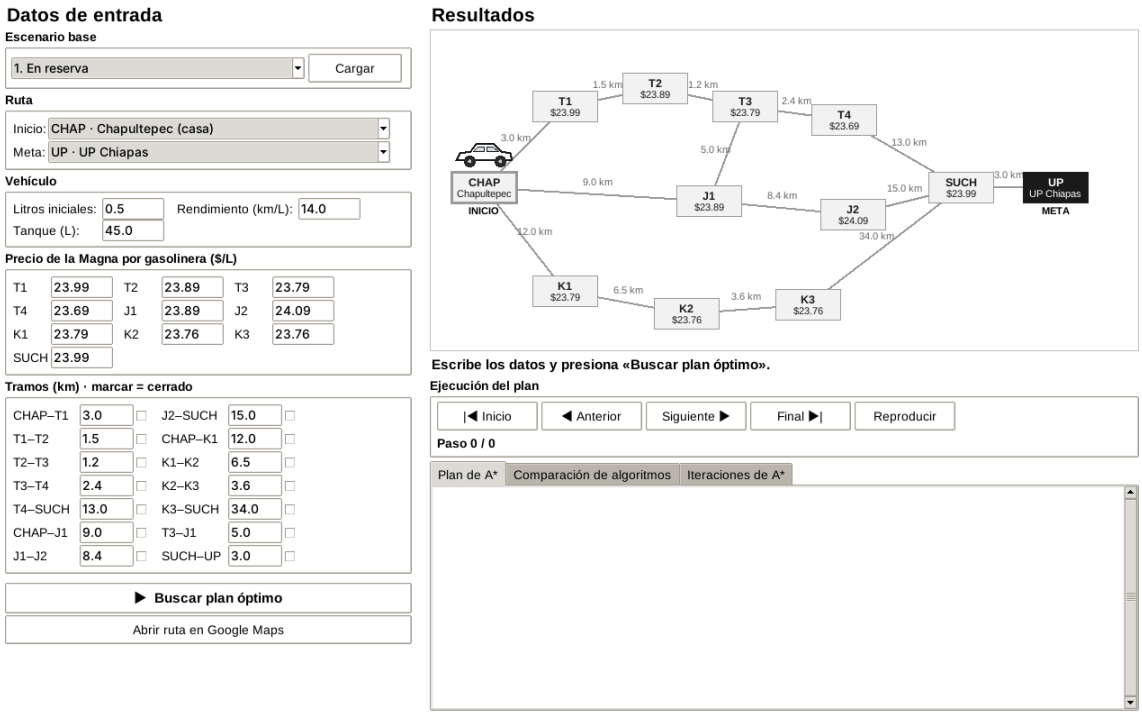


Figura 3.1: Caso 1: especificación del estado inicial y la meta en la interfaz.

3.1.3. Solución

La Figura 3.2 muestra la solución en la interfaz y el Listado 3.1 el plan completo. El agente va por la **ruta Terán**, carga **1 L en Calz. Emiliano Zapata (T3)** y **1 L en Santa Elena (T4)**; gasta **$47.48** y llega con 0.78 L. La Tabla 3.1 compara los cuatro algoritmos.

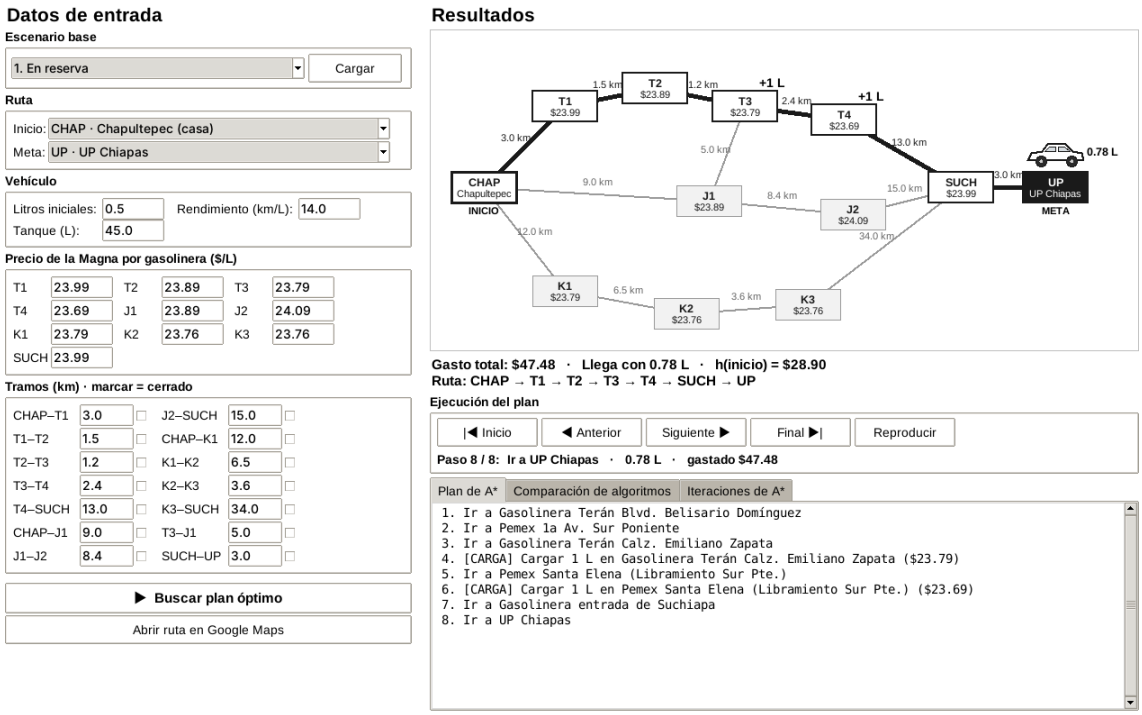


Figura 3.2: Caso 1: solución encontrada por A*.

```
 1. Ir a Gasolinera Terán Blvd. Belisario Domínguez
 2. Ir a Pemex 1a Av. Sur Poniente
 3. Ir a Gasolinera Terán Calz. Emiliano Zapata
 4. [CARGA] Cargar 1 L en Gasolinera Terán Calz. Emiliano Zapata (
    $23.79)
 5. Ir a Pemex Santa Elena (Libramiento Sur Pte.)
 6. [CARGA] Cargar 1 L en Pemex Santa Elena (Libramiento Sur Pte.) (
    $23.69)
 7. Ir a Gasolinera entrada de Suchiapa
 8. Ir a UP Chiapas

Gasto total: $47.48  |  Llega con 0.78 L
Ruta: CHAP → T1 → T2 → T3 → T4 → SUCH → UP
```

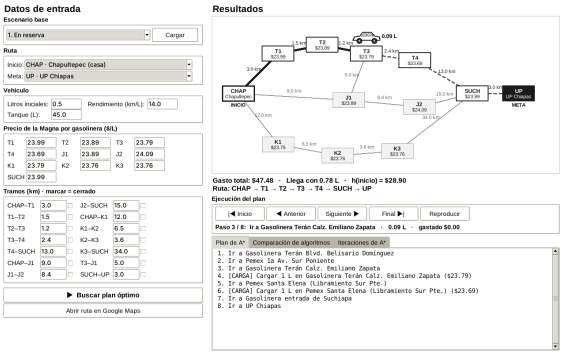
Listing 3.1: Plan del Caso 1.

Tabla 3.1: Comparación de algoritmos, Caso 1.

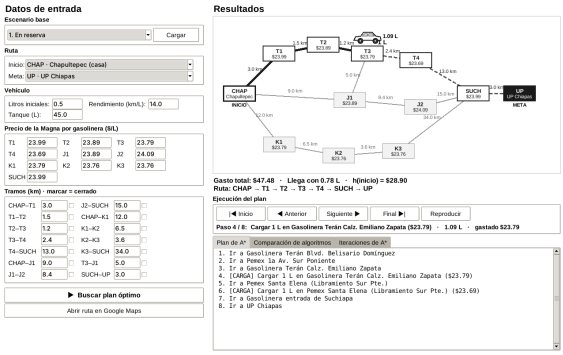
Algoritmo	**Gasto ($)**	**Expandidos**	**Frontera máx.**	**¿Óptimo?**
BFS	47.58	364	819	No
UCS	47.48	263	672	Sí
Greedy	47.98	204	513	No
A*	47.48	121	372	Sí

3.1.4. Ejecución

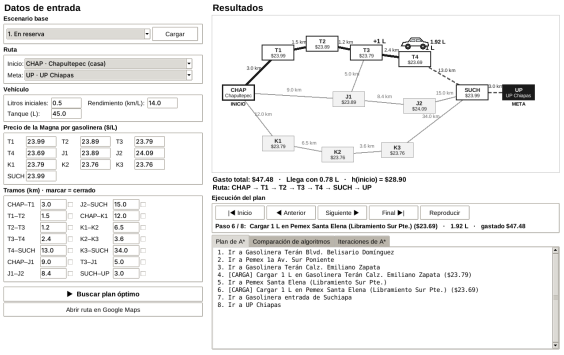
La Figura 3.3 muestra la secuencia de ejecución. En (a) el auto llega a T3 con solo 0.09 L, insuficientes para el tramo a T4 (0.17 L); en (b) carga 1 L; en (c), ya en Santa Elena, carga el segundo litro al precio más barato; en (d) llega a Suchiapa con 0.99 L, suficientes para el último tramo.



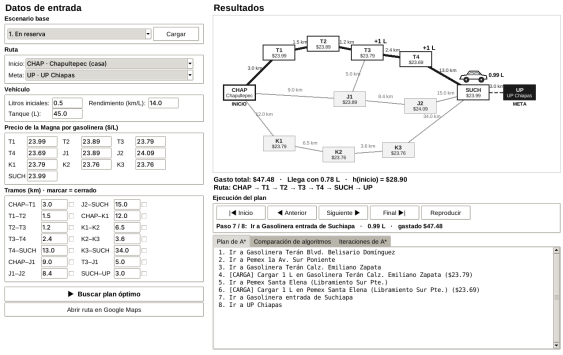
(a) Paso 3/8: Ir a Gasolinera Terán Calz. Emiliano Zapata. Tanque: 0.09 L; gastado: $0.00.



(b) Paso 4/8: Cargar 1 L en Gasolinera Terán Calz. Emiliano Zapata ($23.79). Tanque: 1.09 L; gastado: $23.79.



(c) Paso 6/8: Cargar 1 L en Pemex Santa Elena (Libramiento Sur Pte.) ($23.69). Tanque: 1.92 L; gastado: $47.48.



(d) Paso 7/8: Ir a Gasolinera entrada de Suchiapa. Tanque: 0.99 L; gastado: $47.48.

Figura 3.3: Caso 1: ejecución paso a paso del plan.

3.1.5. Comentarios

- A* y UCS encontraron el mismo plan óptimo, pero A* expandió **121 estados contra 263** de UCS, menos de la mitad. Las 121 iteraciones completas de A* se muestran en el Anexo 5.3.
- BFS cargó 2 L de una vez en T3 porque busca menos acciones, no menos dinero; su plan cuesta 10 centavos más.

- Greedy cargó 2 L en la primera gasolinera (T1, la más cara de la ruta) porque solo mira lo que falta y no lo que ya gastó.
- Repartir la carga entre T3 y T4 no es una decisión obvia: aparece solo porque la búsqueda considera al mismo tiempo la ruta, los litros y los precios.

3.2. Caso 2: Obra en el Libramiento Sur

3.2.1. Descripción

El auto sale con 0.5 L, pero el tramo **Calz. Emiliano Zapata – Santa Elena (T3–T4) está cerrado** por obra. La ruta más corta deja de existir y el agente debe encontrar otro camino sin perder de vista la gasolina.

3.2.2. Estado inicial y meta

Estado inicial: $(\text{CHAP}, 0.5, \text{falso})$. Meta: $(\text{UP}, c \geq 0)$. En la interfaz se cargó el escenario *3. Obra en el Libramiento Sur*, que marca el tramo T3–T4 como cerrado (Figura 3.4). Ahora el camino de menor consumo pasa por El Jobo, los litros mínimos suben a 2.52 L y $h(\text{inicio}) = \$47.85$.

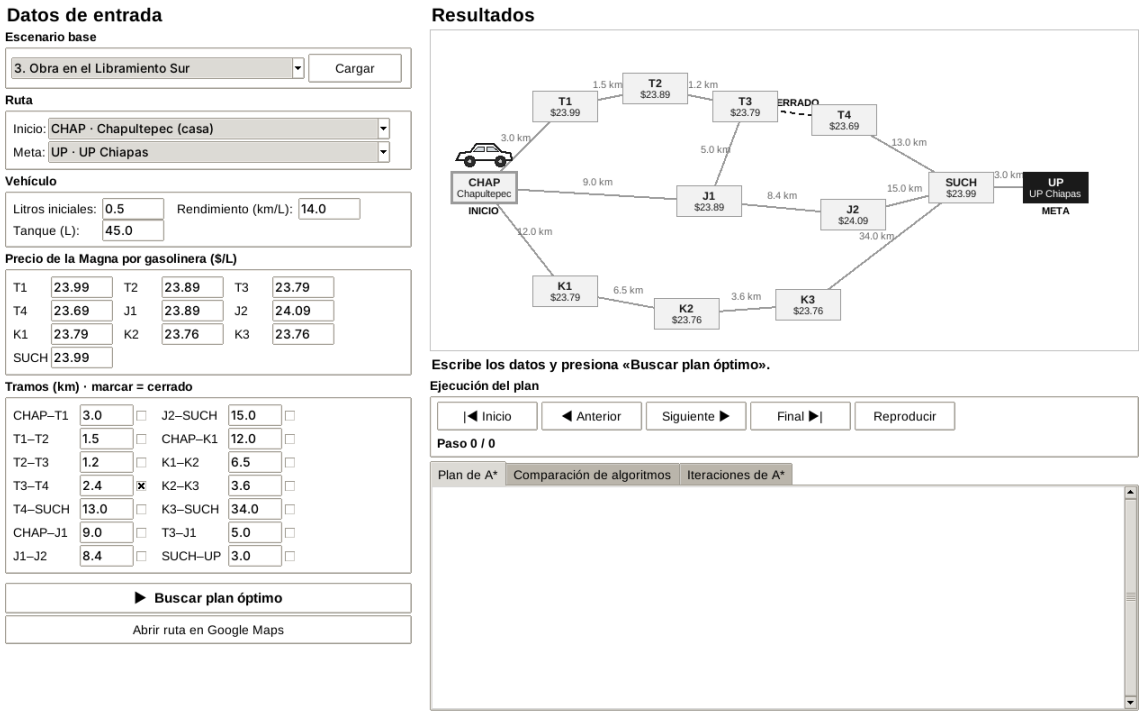


Figura 3.4: Caso 2: especificación con el tramo T3–T4 cerrado.

3.2.3. Solución

La solución (Figura 3.5 y Listado 3.2) empieza por Terán, carga **3 L de una vez en T3** y cruza hacia la ruta El Jobo por Gamboa (J1). Gasta **$71.37** y llega con 0.85 L. La Tabla 3.2 muestra la comparación.

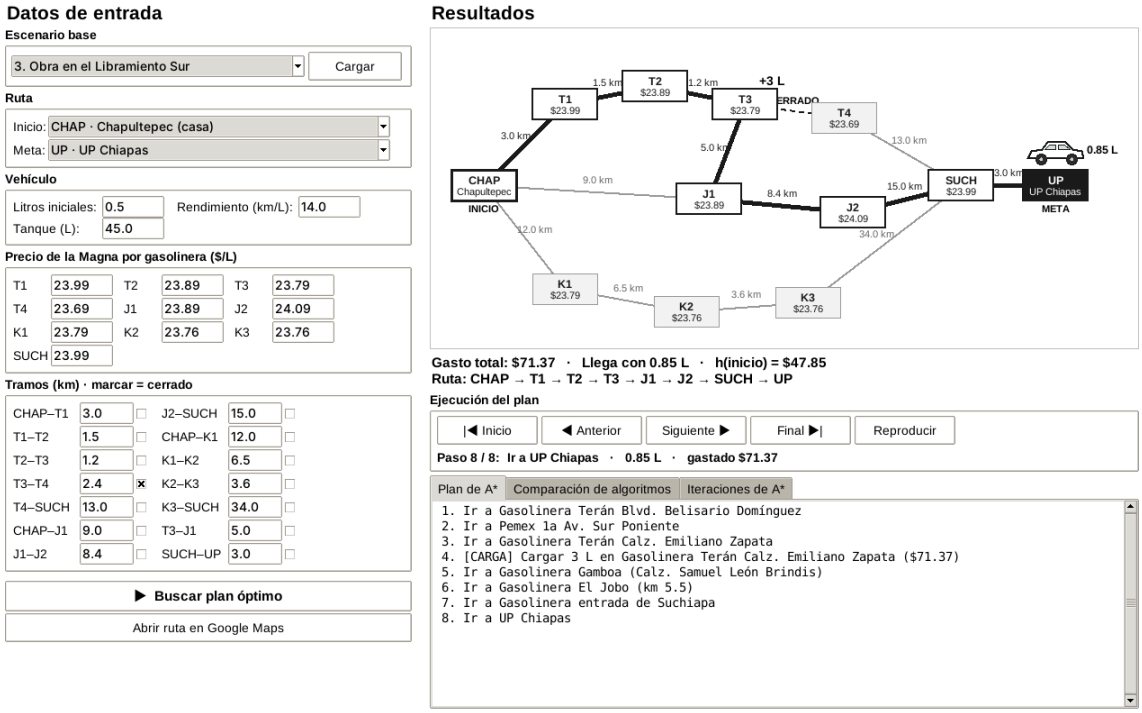


Figura 3.5: Caso 2: solución encontrada por A*.

```
 1. Ir a Gasolinera Terán Blvd. Belisario Domínguez
 2. Ir a Pemex 1a Av. Sur Poniente
 3. Ir a Gasolinera Terán Calz. Emiliano Zapata
 4. [CARGA] Cargar 3 L en Gasolinera Terán Calz. Emiliano Zapata (
    $71.37)
 5. Ir a Gasolinera Gamboa (Calz. Samuel León Brindis)
 6. Ir a Gasolinera El Jobo (km 5.5)
 7. Ir a Gasolinera entrada de Suchiapa
 8. Ir a UP Chiapas

Gasto total: $71.37  |  Llega con 0.85 L
Ruta: CHAP → T1 → T2 → T3 → J1 → J2 → SUCH → UP
```

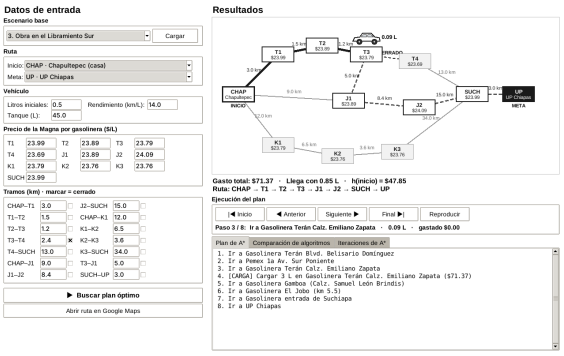
Listing 3.2: Plan del Caso 2.

Tabla 3.2: Comparación de algoritmos, Caso 2.

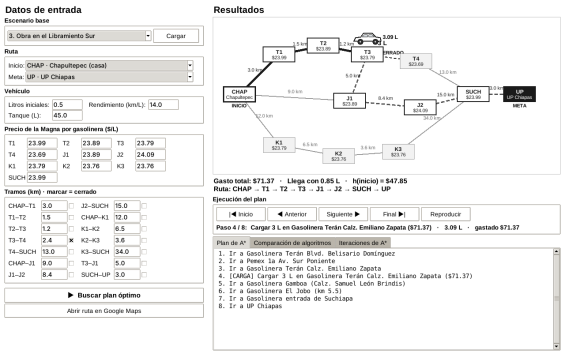
Algoritmo	Gasto ($)	Expandidos	Frontera máx.	¿Óptimo?
BFS	71.97	505	994	No
UCS	71.37	1253	2567	Sí
Greedy	71.97	132	302	No
A*	71.37	335	824	Sí

3.2.4. Ejecución

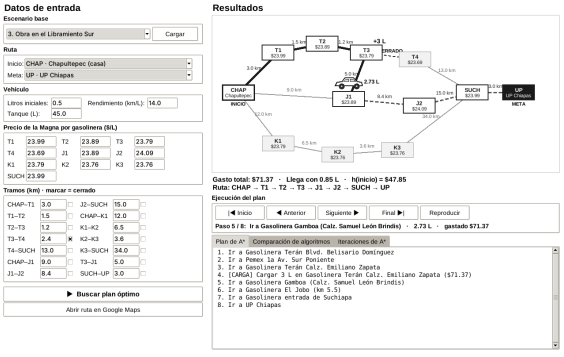
En la Figura 3.6: (a) llega a T3 con 0.09 L; (b) carga 3 L; (c) cruza a Gamboa por el tramo de 5 km; (d) llega a Suchiapa por la ruta El Jobo con 1.06 L.



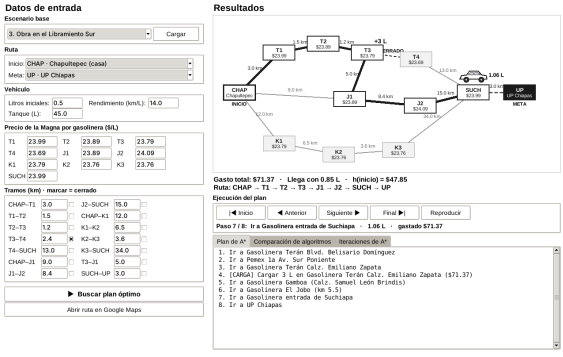
(a) Paso 3/8: Ir a Gasolinera Terán Calz. Emiliano Zapata. Tanque: 0.09 L; gastado: $0.00.



(b) Paso 4/8: Cargar 3 L en Gasolinera Terán Calz. Emiliano Zapata ($71.37). Tanque: 3.09 L; gastado: $71.37.



(c) Paso 5/8: Ir a Gasolinera Gamboa (Calz. Samuel León Brindis). Tanque: 2.73 L; gastado: $71.37.



(d) Paso 7/8: Ir a Gasolinera entrada de Suchiapa. Tanque: 1.06 L; gastado: $71.37.

Figura 3.6: Caso 2: ejecución paso a paso del plan.

3.2.5. Comentarios

- Cargar 2 L no alcanza: desde T3 por El Jobo se necesitan 2.24 L. Cargar 2 L en T3 y 1 L en J1 costaría $71.47, diez centavos más que cargar 3 L en T3.
- Es el caso donde más se nota la heurística: **UCS expandió 1 253 estados y A* solo 335**, casi cuatro veces menos.
- Greedy expandió menos que A* (132) pero dio un plan más caro ($71.97), lo que muestra el equilibrio entre rapidez y calidad de la solución.

3.3. Caso 3: Promoción en Gamboa

3.3.1. Descripción

El auto sale con 0.5 L y la gasolinera **Gamboa (J1) tiene una promoción de $21.49** por litro, mucho más barata que las demás, pero no está en la ruta más corta.

3.3.2. Estado inicial y meta

Estado inicial: $(\text{CHAP}, 0.5, \text{falso})$. Meta: $(\text{UP}, c \geq 0)$. Se cargó el escenario *4. Promoción en El Jobo* (Figura 3.7), donde el precio de J1 aparece en $21.49. Como ahora el precio más barato es $21.49, $h(\text{inicio}) = (1.72 - 0.5) \times 21.49 = \$26.22$.

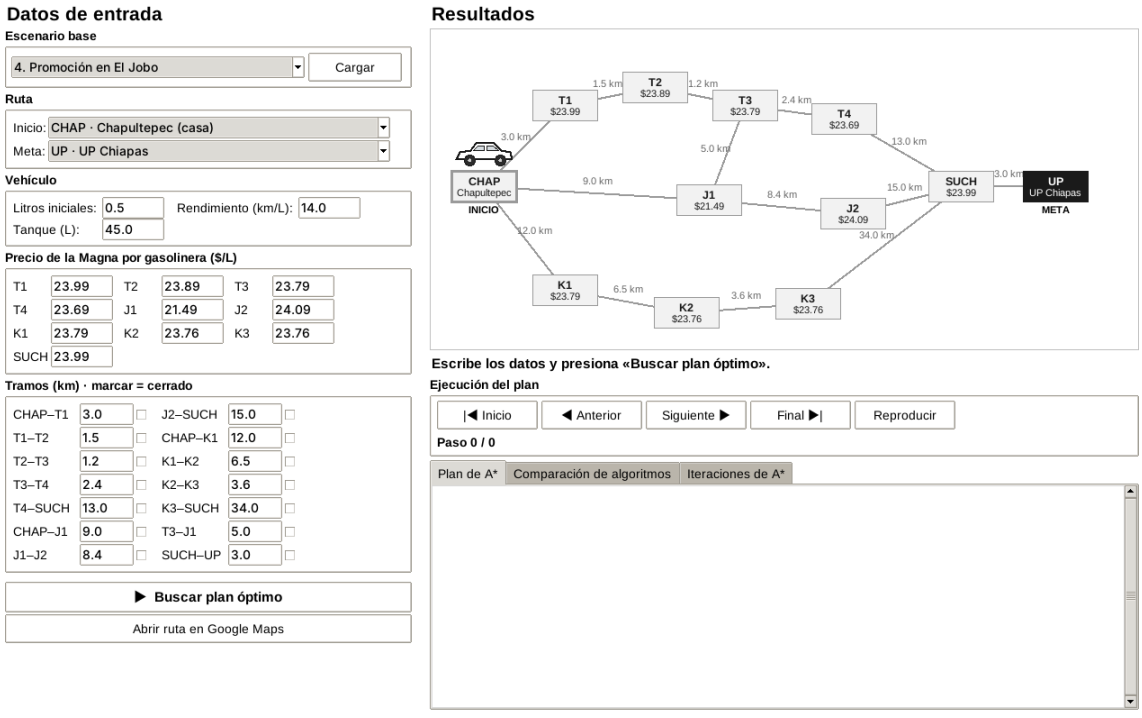


Figura 3.7: Caso 3: especificación con la promoción en J1.

3.3.3. Solución

El agente carga 1 L en T3 para poder moverse, **se desvía 5 km a Gamboa**, carga 1 L en promoción, **regresa** a T3 y sigue por Santa Elena (Figura 3.8 y Listado 3.3). Gasta **$45.28**, $2.20 menos que en el Caso 1. La Tabla 3.3 muestra la comparación.

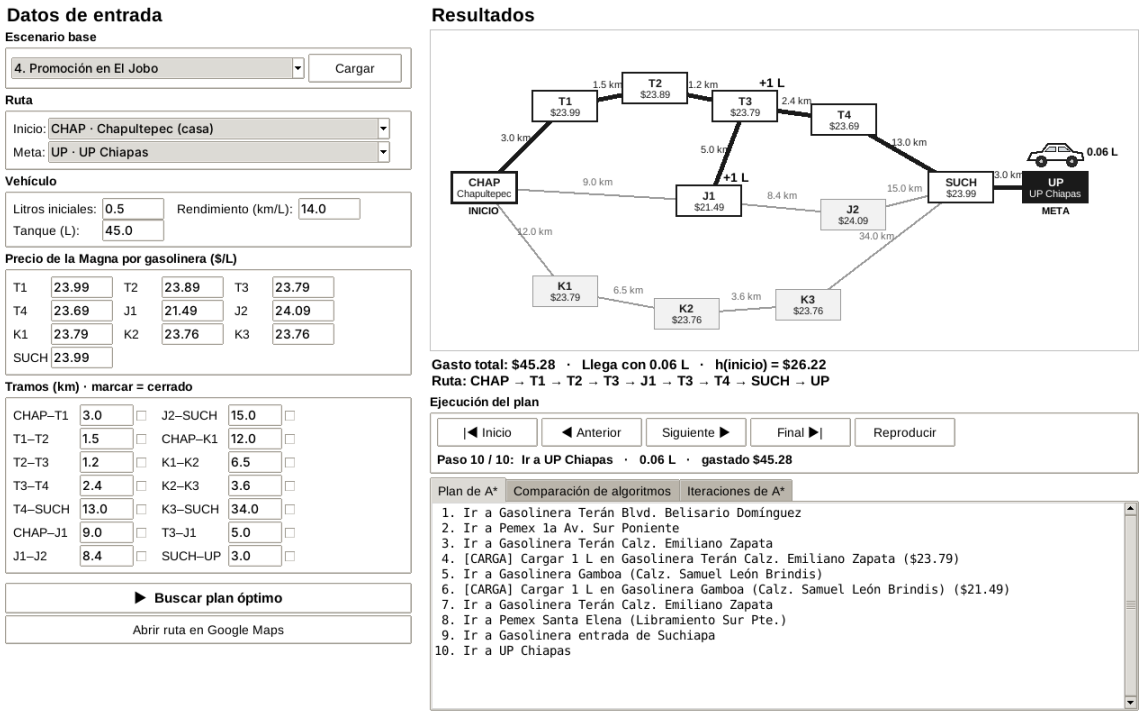


Figura 3.8: Caso 3: solución encontrada por A*.

```
 1. Ir a Gasolinera Terán Blvd. Belisario Domínguez
 2. Ir a Pemex 1a Av. Sur Poniente
 3. Ir a Gasolinera Terán Calz. Emiliano Zapata
 4. [CARGA] Cargar 1 L en Gasolinera Terán Calz. Emiliano Zapata (
    $23.79)
 5. Ir a Gasolinera Gamboa (Calz. Samuel León Brindis)
 6. [CARGA] Cargar 1 L en Gasolinera Gamboa (Calz. Samuel León
    Brindis) ($21.49)
 7. Ir a Gasolinera Terán Calz. Emiliano Zapata
 8. Ir a Pemex Santa Elena (Libramiento Sur Pte.)
 9. Ir a Gasolinera entrada de Suchiapa
10. Ir a UP Chiapas

Gasto total: $45.28  |  Llega con 0.06 L
Ruta: CHAP → T1 → T2 → T3 → J1 → T3 → T4 → SUCH → UP
```

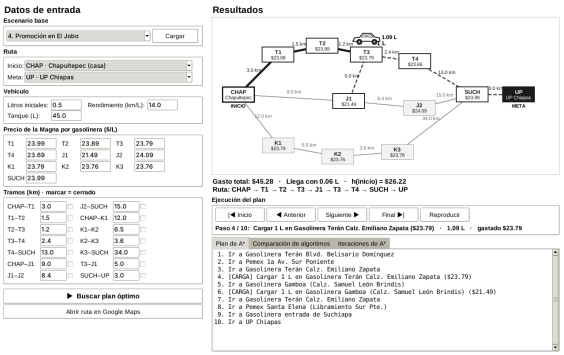
Listing 3.3: Plan del Caso 3.

Tabla 3.3: Comparación de algoritmos, Caso 3.

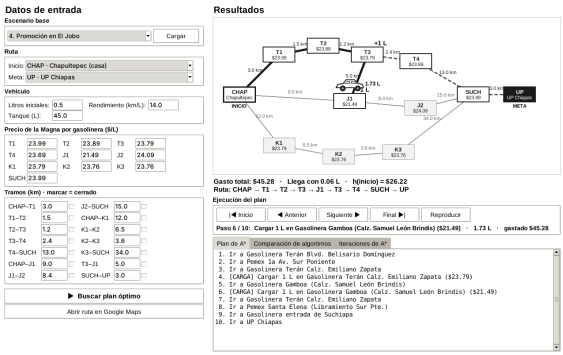
Algoritmo	Gasto ($)	Expandidos	Frontera máx.	¿Óptimo?
BFS	47.58	364	819	No
UCS	45.28	333	810	Sí
Greedy	47.98	204	513	No
A*	45.28	90	299	Sí

3.3.4. Ejecución

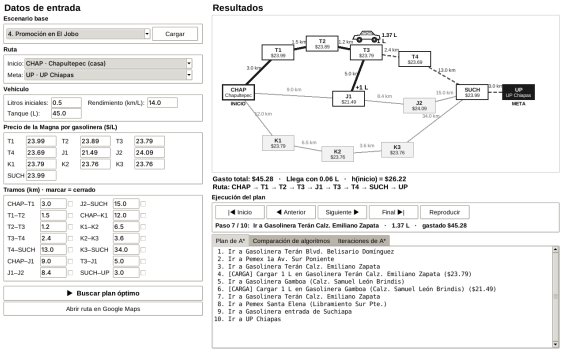
En la Figura 3.9: (a) carga 1 L en T3; (b) ya en Gamboa, carga 1 L a $21.49; (c) regresa a T3 con 1.37 L; (d) llega a Suchiapa con 0.27 L, suficientes para los 0.21 L del último tramo.



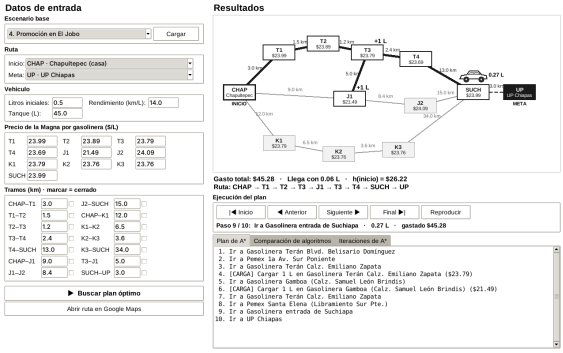
(a) Paso 4/10: Cargar 1 L en Gasolinera Terán Calz. Emiliano Zapata ($23.79). Tanque: 1.09 L; gastado: $23.79.



(b) Paso 6/10: Cargar 1 L en Gasolinera Gamboa (Calz. Samuel León Brindis) ($21.49). Tanque: 1.73 L; gastado: $45.28.



(c) Paso 7/10: Ir a Gasolinera Terán Calz. Emiliano Zapata. Tanque: 1.37 L; gastado: $45.28.



(d) Paso 9/10: Ir a Gasolinera entrada de Suchiapa. Tanque: 0.27 L; gastado: $45.28.

Figura 3.9: Caso 3: ejecución paso a paso del plan.

3.3.5. Comentarios

- El desvío de ida y vuelta consume 0.72 L, pero el ahorro en el precio lo compensa; el agente llega con apenas 0.06 L, lo cual es válido según las reglas.
- BFS y Greedy no aprovechan la promoción y dan los mismos planes que en el Caso 1.
- A* expandió solo **90 estados contra 333** de UCS.

3.4. Caso 4: Datos ingresados por el usuario

3.4.1. Descripción

Este caso no está predefinido en el código: se armó **escribiendo los datos en la interfaz**, como lo haría cualquier usuario. A partir del escenario 1 se cambiaron los datos de la Tabla 3.4, que simulan un día con tráfico, cambios de precio y una obra.

Tabla 3.4: Datos de entrada modificados en el Caso 4.

Dato	Original	Ingresado	Motivo
Litros iniciales	0.5 L	**0.8 L**	Sale con un poco más de gasolina.
Rendimiento	14 km/L	**12 km/L**	Tráfico y aire acondicionado.
Precio T4	\$23.69	**\$24.29**	Sube la que era la más barata.
Precio K2	\$23.76	**\$22.49**	Promoción en la ruta más larga.
Tramo J1–J2	Abierto	**Cerrado**	Obra en la ruta El Jobo.

3.4.2. Estado inicial y meta

Estado inicial: $(\text{CHAP}, 0.8, \text{falso})$. Meta: $(\text{UP}, c \geq 0)$. La Figura 3.10 muestra los datos escritos en la interfaz. Con 12 km/L los litros mínimos suben a 2.00 L y el precio más barato es \$22.49, así que $h(\text{inicio}) = (2.00 - 0.8) \times 22.49 = \$26.99$.

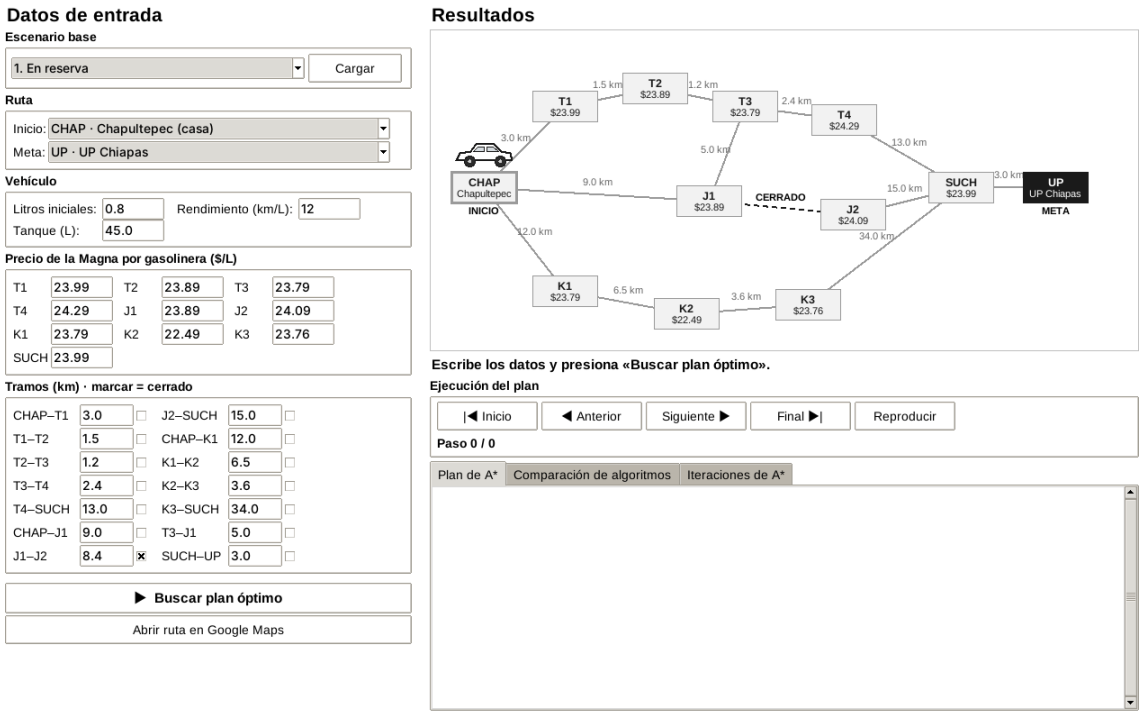


Figura 3.10: Caso 4: datos escritos por el usuario en la interfaz.

3.4.3. Solución

El agente va por Terán y carga **2 L de una vez en T3**; gasta **$47.58** y llega con 0.80 L (Figura 3.11 y Listado 3.4). La Tabla 3.5 muestra la comparación y la Figura 3.12 la pestaña de iteraciones de A*.

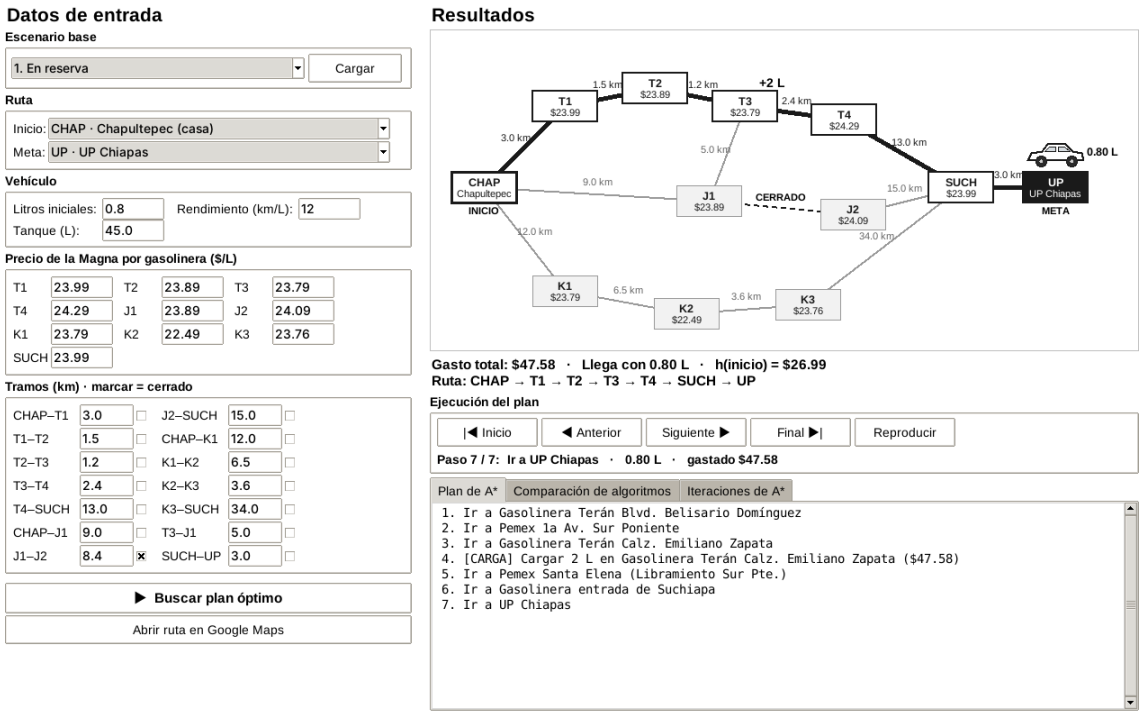


Figura 3.11: Caso 4: solución encontrada por A*.

```
 1. Ir a Gasolinera Terán Blvd. Belisario Domínguez
 2. Ir a Pemex 1a Av. Sur Poniente
 3. Ir a Gasolinera Terán Calz. Emiliano Zapata
 4. [CARGA] Cargar 2 L en Gasolinera Terán Calz. Emiliano Zapata (
    $47.58)
 5. Ir a Pemex Santa Elena (Libramiento Sur Pte.)
 6. Ir a Gasolinera entrada de Suchiapa
 7. Ir a UP Chiapas

Gasto total: $47.58  |  Llega con 0.80 L
Ruta: CHAP → T1 → T2 → T3 → T4 → SUCH → UP
```

Listing 3.4: Plan del Caso 4.

Tabla 3.5: Comparación de algoritmos, Caso 4.

Algoritmo	**Gasto ($)**	**Expandidos**	**Frontera máx.**	**¿Óptimo?**
BFS	47.78	330	637	No
UCS	47.58	350	847	Sí
Greedy	47.98	176	393	No
A*	47.58	132	371	Sí

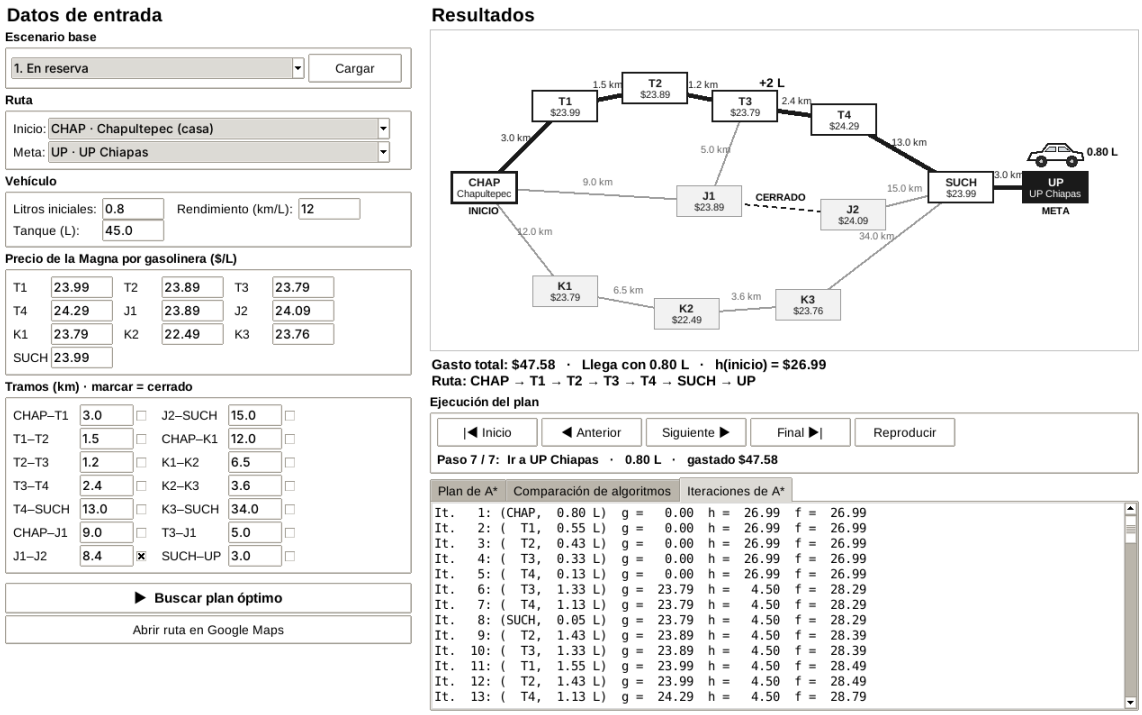
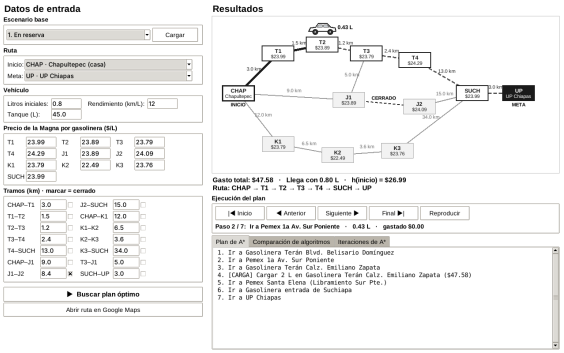


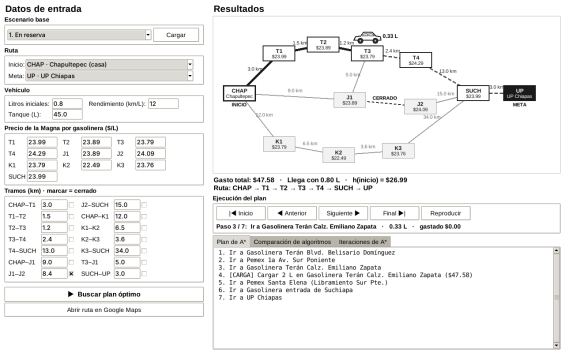
Figura 3.12: Caso 4: iteraciones de A* mostradas en la interfaz.

3.4.4. Ejecución

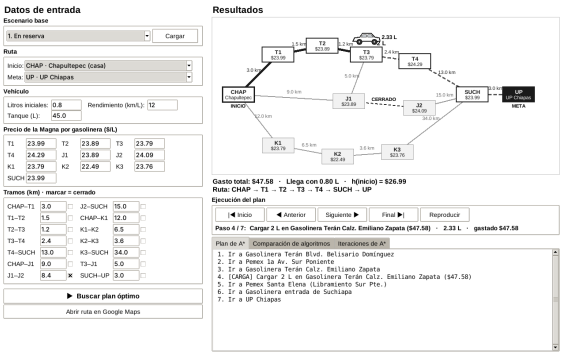
En la Figura 3.13: (a) llega a T2 con 0.43 L; (b) llega a T3 con 0.33 L; (c) carga 2 L; (d) llega a Suchiapa con 1.05 L.



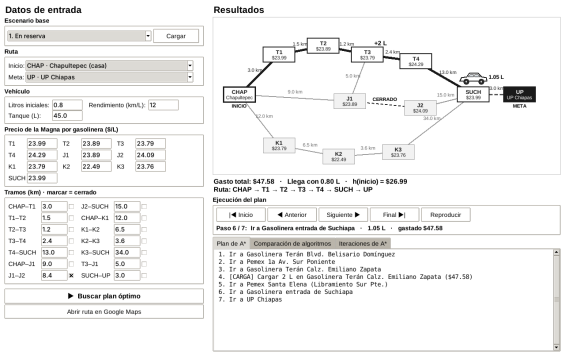
(a) Paso 2/7: Ir a Pemex 1a Av. Sur Poniente. Tanque: 0.43 L; gastado: $0.00.



(b) Paso 3/7: Ir a Gasolinera Terán Calz. Emiliano Zapata. Tanque: 0.33 L; gastado: $0.00.



(c) Paso 4/7: Cargar 2 L en Gasolinera Terán Calz. Emiliano Zapata ($47.58). Tanque: 2.33 L; gastado: $47.58.



(d) Paso 6/7: Ir a Gasolinera entrada de Suchiapa. Tanque: 1.05 L; gastado: $47.58.

Figura 3.13: Caso 4: ejecución paso a paso del plan.

3.4.5. Comentarios

- Como Santa Elena subió a $24.29, ya no conviene repartir la carga como en el Caso 1: 2 L en T3 ($47.58) es más barato que 1 L en T3 y 1 L en T4 ($48.08).
- La promoción de K2 no se aprovecha porque está en la ruta de 59.1 km: con 12 km/L habría que comprar más de 4 L extra solo para recorrerla.
- En las iteraciones (Figura 3.12) se ve que A* primero prueba cargar solo 1 L, pero llega a Suchiapa con 0.05 L, que no alcanzan para el último tramo (0.25 L); esa rama no tiene salida y termina eligiendo 2 L. A* expandió **132 estados contra 350** de UCS.

3.5. Caso 5: Regreso a casa (cambio de inicio y meta)

3.5.1. Descripción

Este caso demuestra que desde la interfaz se pueden cambiar el inicio y la meta. Ahora el auto está en la **UP Chiapas con 0.3 L** y quiere **regresar a Chapultepec**. Además, la gasolinera de la entrada de Suchiapa sube a **$24.49**, y es la única que se alcanza con esa gasolina.

3.5.2. Estado inicial y meta

Estado inicial: $(\text{UP}, 0.3, \text{falso})$. Meta: $(\text{CHAP}, c \geq 0)$. En la interfaz se eligió *UP* como inicio y *CHAP* como meta (Figura 3.14). Al cambiar la meta, la heurística se recalcula desde Chapultepec: $h(\text{inicio}) = (1.72 - 0.3) \times 23.69 = \$33.64$.

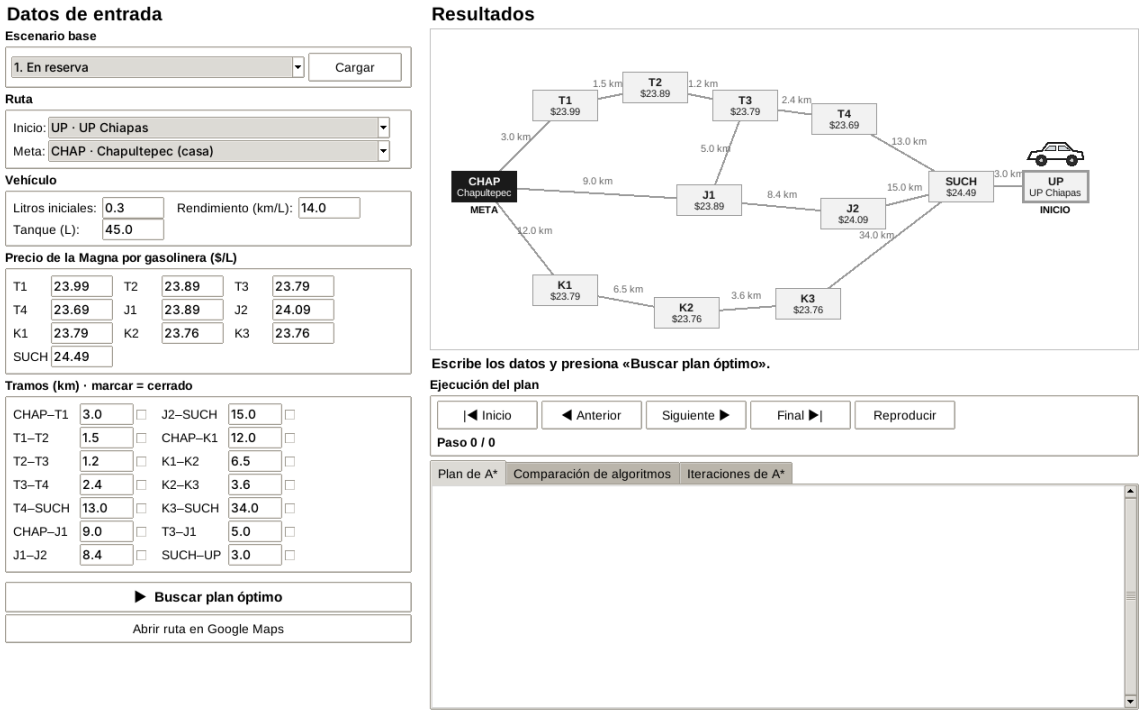


Figura 3.14: Caso 5: especificación con inicio en la UP y meta en Chapultepec.

3.5.3. Solución

El agente carga **solo 1 L en Suchiapa** (lo mínimo para llegar a Santa Elena, porque es cara) y **1 L en Santa Elena**, la más barata; después sigue por Terán hasta casa. Gasta **$48.18** y llega con 0.58 L (Figura 3.15 y Listado 3.5). La Tabla 3.6 muestra la comparación.

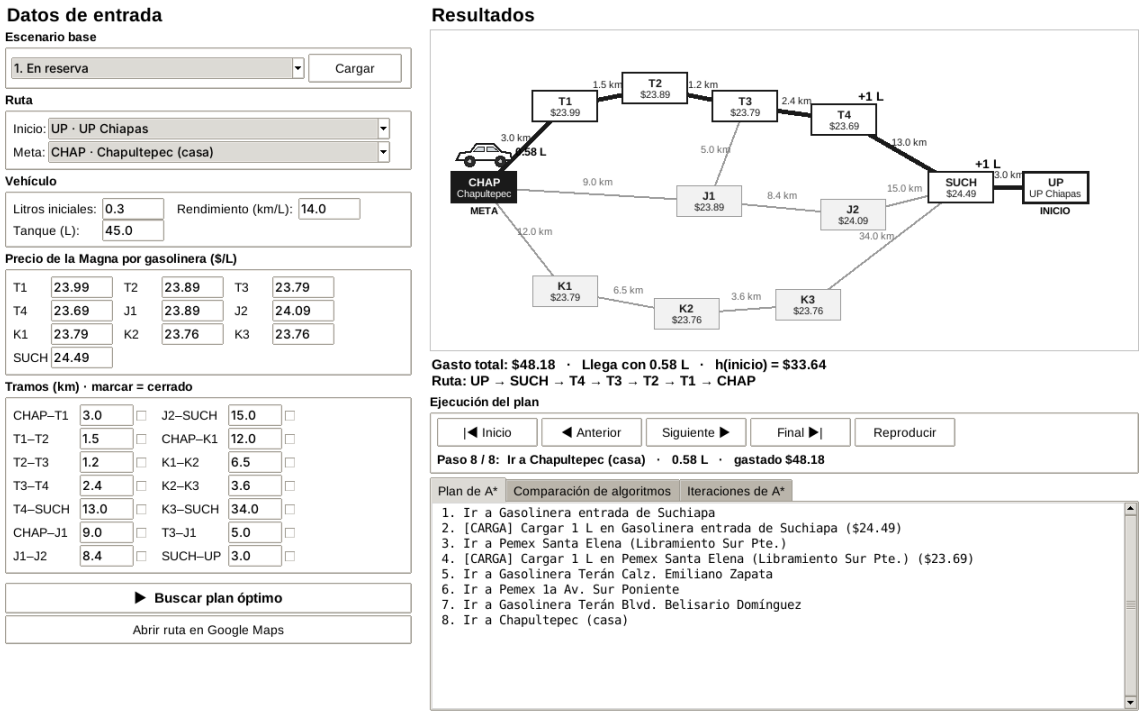


Figura 3.15: Caso 5: solución encontrada por A*.

```
 1. Ir a Gasolinera entrada de Suchiapa
 2. [CARGA] Cargar 1 L en Gasolinera entrada de Suchiapa ($24.49)
 3. Ir a Pemex Santa Elena (Libramiento Sur Pte.)
 4. [CARGA] Cargar 1 L en Pemex Santa Elena (Libramiento Sur Pte.) (
    $23.69)
 5. Ir a Gasolinera Terán Calz. Emiliano Zapata
 6. Ir a Pemex 1a Av. Sur Poniente
 7. Ir a Gasolinera Terán Blvd. Belisario Domínguez
 8. Ir a Chapultepec (casa)

Gasto total: $48.18  |  Llega con 0.58 L
Ruta: UP → SUCH → T4 → T3 → T2 → T1 → CHAP
```

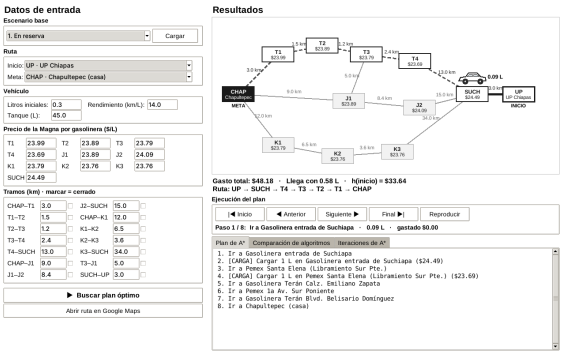
Listing 3.5: Plan del Caso 5.

Tabla 3.6: Comparación de algoritmos, Caso 5.

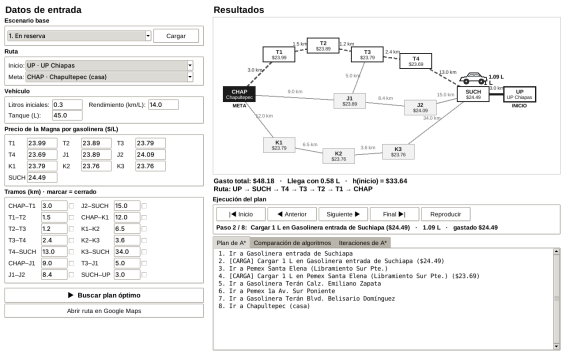
Algoritmo	**Gasto ($)**	**Expandidos**	**Frontera máx.**	**¿Óptimo?**
BFS	73.47	80	191	No
UCS	48.18	23	53	Sí
Greedy	73.47	41	119	No
A*	48.18	14	37	Sí

3.5.4. Ejecución

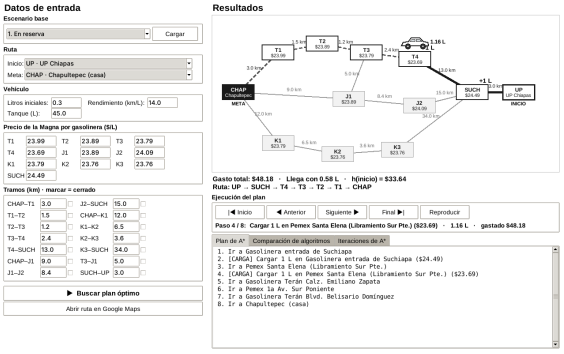
En la Figura 3.16: (a) llega a Suchiapa con 0.09 L; (b) carga 1 L a \$24.49; (c) en Santa Elena carga 1 L a \$23.69; (d) va por T2 con 0.90 L, suficientes para llegar a casa.



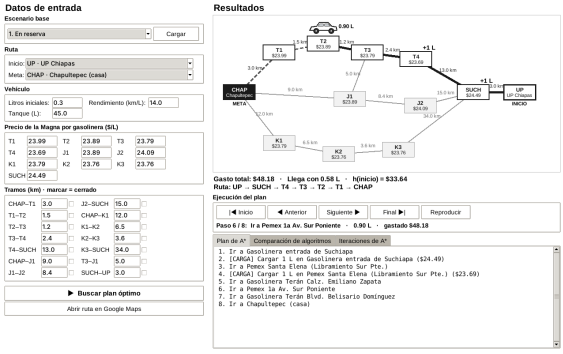
(a) Paso 1/8: Ir a Gasolinera entrada de Suchiapa. Tanque: 0.09 L; gastado: \$0.00.



(b) Paso 2/8: Cargar 1 L en Gasolinera entrada de Suchiapa (\$24.49). Tanque: 1.09 L; gastado: \$24.49.



(c) Paso 4/8: Cargar 1 L en Pemex Santa Elena (Libramiento Sur Pte.) (\$23.69). Tanque: 1.16 L; gastado: \$48.18.



(d) Paso 6/8: Ir a Pemex 1a Av. Sur Poniente. Tanque: 0.90 L; gastado: \$48.18.

Figura 3.16: Caso 5: ejecución paso a paso del plan.

3.5.5. Comentarios

- La diferencia entre algoritmos es la más grande de todos los casos: BFS y Greedy cargaron 3 L en Suchiapa (\$73.47), **\$25.29 más** que el plan óptimo.
- El agente entendió que, aunque está obligado a cargar en Suchiapa, debe cargar lo mínimo ahí y completar en la gasolinera más barata.
- A* expandió **14 estados contra 23** de UCS. El espacio explorado es pequeño porque con tan poca gasolina hay muy pocas ramas válidas.

3.6. Comparación general

La Figura 3.17 muestra los estados expandidos por cada algoritmo en los cinco casos y la Tabla 3.7 resume la solución de A* en cada uno.

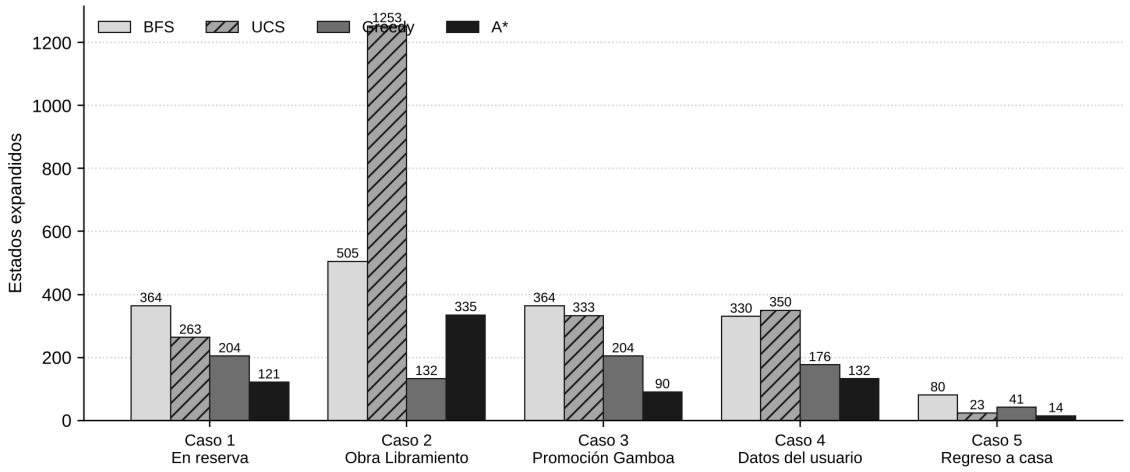


Figura 3.17: Estados expandidos por algoritmo en los cinco casos.

Tabla 3.7: Resumen de los cinco casos resueltos con A*.

Caso	**Ruta**	**Cargas**	**Gasto**	**A***	**UCS**
1. En reserva	Terán	1 L T3 + 1 L T4	$47.48	121	263
2. Obra Libramiento	Terán → El Jobo	3 L T3	$71.37	335	1 253
3. Promoción Gamboa	Terán con desvío	1 L T3 + 1 L J1	$45.28	90	333
4. Datos del usuario	Terán	2 L T3	$47.58	132	350
5. Regreso a casa	Terán (inversa)	1 L SUCH + 1 L T4	$48.18	14	23

En resumen, A* encontró el óptimo en los cinco casos expandiendo entre 1.6 y 3.7 veces menos estados que UCS, mientras que BFS y Greedy nunca encontraron el plan más barato.

Capítulo 4

Conclusiones y recomendaciones

4.1. Conclusiones

Se logró modelar el traslado entre Chapultepec y la UP Chiapas como un problema de búsqueda en el espacio de estados *(ubicación, litros, bandera de carga)* y resolverlo con un programa en Python que compara BFS, UCS, Greedy y A*. En los cinco casos del Capítulo de Resultados, **A* con una heurística admisible encontró siempre el plan de menor costo** y lo hizo expandiendo entre 1.6 y 3.7 veces menos estados que UCS (Tabla 3.7), mientras que BFS y Greedy entregaron planes más caros.

¿Qué se aprendió?

- Que lo más importante es **diseñar bien el estado**: si el estado no incluyera los litros del tanque, el agente no sabría si alcanza a llegar a una gasolinera, y sin la bandera de carga podría quedarse cargando indefinidamente en el mismo lugar.
- Que una heurística debe ser **admisible** para que A* sea óptimo. Nuestra primera idea (distancia en línea recta $\times$ consumo $\times$ precio) ignoraba la gasolina del tanque y podía sobreestimar; la corregimos usando los litros que realmente faltan (Sección 1.9).
- Que la búsqueda debe terminar al **sacar** la meta de la frontera y no al generarla; de lo contrario UCS y A* pueden devolver un plan que no es el más barato.
- Que separar el método de búsqueda de la visualización facilita probarlo: la consola y la interfaz usan exactamente el mismo código (Tabla 2.1).

¿Qué haríamos diferente?

- Usaríamos dos criterios desde el inicio (dinero y distancia o tiempo). Con el criterio de solo dinero, cuando el auto no necesita cargar cualquier ruta cuesta $0 y A* puede elegir una ruta más larga (Anexo 5.2).
- Daríamos valor a la gasolina sobrante al llegar, porque hoy un plan que llega con 0.06 L (Caso 3) se considera igual de bueno que uno que llega con 0.80 L si cuestan lo mismo.

- Separaríamos desde el principio el código en módulos; al inicio todo estaba en un solo archivo y tuvimos que reorganizarlo.

¿Qué nuevos proyectos se podrían emprender?

- Planear la carga de gasolina de **toda la semana**, con precios que cambian por día y varios viajes.
- Una aplicación móvil que tome los precios reales de las gasolineras y la ubicación del GPS para recalcular el plan en tiempo real.
- Extender el modelo a rutas de reparto con varios vehículos (un problema multiagente).

4.2. Recomendaciones

- **Datos reales en línea:** integrar una fuente de precios actualizada de gasolina y la API de Google Maps [2] para obtener distancias y tiempos con tráfico, en lugar de capturarlos a mano.
- **Costo combinado:** agregar el tiempo de recorrido o la distancia como segundo criterio, por ejemplo con una suma ponderada, cuidando que la heurística siga siendo admisible.
- **Cargas flexibles:** permitir cargas de cualquier monto (por ejemplo, en pesos) en lugar de solo 1, 2 o 3 litros.
- **Interfaz:** mostrar en el mapa el árbol de búsqueda o los nodos expandidos de cada algoritmo, y permitir guardar y cargar escenarios en archivos.
- **Pruebas:** agregar pruebas automáticas que verifiquen que UCS y A* siempre dan el mismo costo en escenarios generados al azar.

Capítulo 5

Anexos

5.1. Anexo A: Cómo ejecutar el programa

El programa se compone de los tres archivos de la Tabla 2.1, que deben estar en la misma carpeta. Solo se requiere Python 3; la librería `matplotlib` es opcional y solo se usa para la gráfica del modo consola.

1. **Interfaz visual:** ejecutar `python interfaz.py`, elegir un escenario base o escribir los datos (inicio, meta, litros, rendimiento, tanque, precios, km y tramos cerrados) y presionar *Buscar plan óptimo*.

2. **Modo consola:** ejecutar `python carga_inteligente.py` y elegir un escenario del menú, o `python carga_inteligente.py --todos` para ejecutar los cuatro escenarios predefinidos.

5.2. Anexo B: Caso trivial (tanque suficiente)

Con el escenario *2. Tanque suficiente* el auto sale de Chapultepec con 5 L. Como los litros mínimos para llegar son 1.72 L, $h(\text{inicio}) = 0$ y ningún algoritmo necesita cargar: el plan óptimo cuesta \$0.00. La Figura 5.1 muestra que A* eligió la ruta El Jobo; esto ocurre porque, con el criterio de solo dinero, todas las rutas sin carga cuestan lo mismo y cualquiera es óptima. Es una limitación que se comenta en las conclusiones. En este caso A* expandió 51 estados, UCS 54, Greedy 138 y BFS 141.

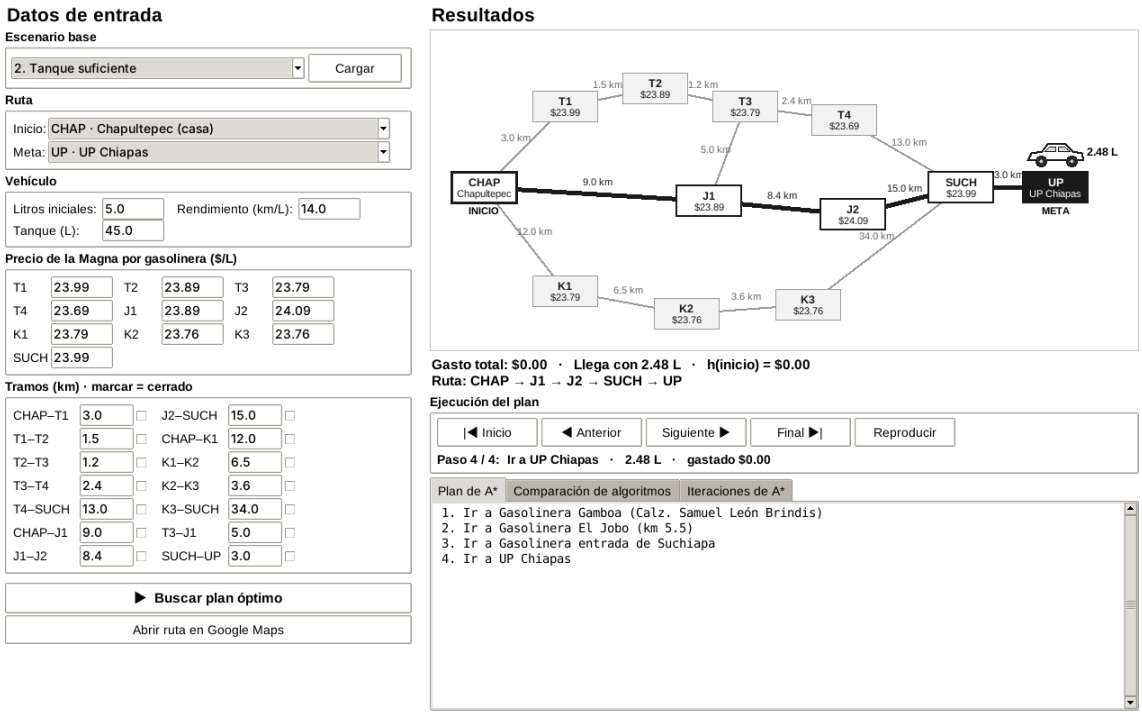


Figura 5.1: Caso trivial: el auto llega sin cargar gasolina.

5.3. Anexo C: Iteraciones completas de A* en el Caso 1

El Listado 5.1 muestra las 121 iteraciones de A* en el Caso 1. En cada línea aparece el estado que se sacó de la frontera, con su lugar y sus litros, y los valores g (dinero gastado), h (estimación de lo que falta) y $f = g + h$. Se observa que f nunca disminuye y que la búsqueda termina en la iteración 121, cuando saca de la frontera el estado meta con $f = \$47.48$.

```
It.   1: (CHAP,  0.50 L)  g =   0.00  h =  28.90  f =  28.90
It.   2: (  T1,  0.29 L)  g =   0.00  h =  28.90  f =  28.90
It.   3: (  T2,  0.18 L)  g =   0.00  h =  28.90  f =  28.90
It.   4: (  T3,  0.09 L)  g =   0.00  h =  28.90  f =  28.90
It.   5: (  T3,  1.09 L)  g =  23.79  h =   5.21  f =  29.00
It.   6: (  T4,  0.92 L)  g =  23.79  h =   5.21  f =  29.00
It.   7: (  T2,  1.18 L)  g =  23.89  h =   5.21  f =  29.10
It.   8: (  T3,  1.09 L)  g =  23.89  h =   5.21  f =  29.10
It.   9: (  T1,  1.29 L)  g =  23.99  h =   5.21  f =  29.20
It.  10: (  T2,  1.18 L)  g =  23.99  h =   5.21  f =  29.20
It.  11: (  T2,  0.00 L)  g =   0.00  h =  33.17  f =  33.17
It.  12: (  T2,  1.00 L)  g =  23.79  h =   9.48  f =  33.27
It.  13: (  T3,  0.91 L)  g =  23.79  h =   9.48  f =  33.27
It.  14: (  T4,  0.74 L)  g =  23.79  h =   9.48  f =  33.27
It.  15: (  T2,  1.00 L)  g =  23.89  h =   9.48  f =  33.37
It.  16: (  T1,  0.07 L)  g =   0.00  h =  34.11  f =  34.11
It.  17: (  T1,  1.07 L)  g =  23.89  h =  10.42  f =  34.31
It.  18: (  T2,  0.96 L)  g =  23.89  h =  10.42  f =  34.31
It.  19: (  T3,  0.87 L)  g =  23.89  h =  10.42  f =  34.31
```

```
It.  20: (  T4,  0.70 L)  g =  23.89  h =  10.42  f =  34.31
It.  21: (  T1,  1.07 L)  g =  23.99  h =  10.42  f =  34.41
It.  22: (  T3,  0.75 L)  g =  23.79  h =  13.27  f =  37.06
It.  23: (  T4,  0.58 L)  g =  23.79  h =  13.27  f =  37.06
It.  24: (  T2,  0.82 L)  g =  23.79  h =  13.74  f =  37.53
It.  25: (  T3,  0.73 L)  g =  23.79  h =  13.74  f =  37.53
It.  26: (  T4,  0.56 L)  g =  23.79  h =  13.74  f =  37.53
It.  27: (  T1,  0.89 L)  g =  23.79  h =  14.69  f =  38.48
It.  28: (  T2,  0.78 L)  g =  23.79  h =  14.69  f =  38.48
It.  29: (  T3,  0.69 L)  g =  23.79  h =  14.69  f =  38.48
It.  30: (  T4,  0.52 L)  g =  23.79  h =  14.69  f =  38.48
It.  31: (CHAP,  0.08 L)  g =   0.00  h =  38.85  f =  38.85
It.  32: (CHAP,  1.08 L)  g =  23.99  h =  15.16  f =  39.15
It.  33: (  T1,  0.87 L)  g =  23.99  h =  15.16  f =  39.15
It.  34: (  T2,  0.76 L)  g =  23.99  h =  15.16  f =  39.15
It.  35: (  T3,  0.67 L)  g =  23.99  h =  15.16  f =  39.15
It.  36: (  T4,  0.50 L)  g =  23.99  h =  15.16  f =  39.15
It.  37: (  T1,  0.85 L)  g =  23.89  h =  15.64  f =  39.53
It.  38: (  T2,  0.74 L)  g =  23.89  h =  15.64  f =  39.53
It.  39: (  T3,  0.65 L)  g =  23.89  h =  15.64  f =  39.53
It.  40: (  T4,  0.48 L)  g =  23.89  h =  15.64  f =  39.53
It.  41: (  T3,  0.57 L)  g =  23.79  h =  17.53  f =  41.32
It.  42: (  T2,  0.66 L)  g =  23.79  h =  17.53  f =  41.32
It.  43: (  T4,  0.40 L)  g =  23.79  h =  17.53  f =  41.32
It.  44: (  T2,  0.64 L)  g =  23.79  h =  18.00  f =  41.79
It.  45: (  T3,  0.55 L)  g =  23.79  h =  18.00  f =  41.79
It.  46: (  T4,  0.38 L)  g =  23.79  h =  18.00  f =  41.79
It.  47: (  T3,  0.53 L)  g =  23.89  h =  18.48  f =  42.37
It.  48: (  T4,  0.36 L)  g =  23.89  h =  18.48  f =  42.37
It.  49: (  T1,  0.71 L)  g =  23.79  h =  18.95  f =  42.74
It.  50: (  T2,  0.60 L)  g =  23.79  h =  18.95  f =  42.74
It.  51: (  T3,  0.51 L)  g =  23.79  h =  18.95  f =  42.74
It.  52: (  T4,  0.34 L)  g =  23.79  h =  18.95  f =  42.74
It.  53: (  T2,  0.58 L)  g =  23.99  h =  19.43  f =  43.42
It.  54: (  T3,  0.49 L)  g =  23.99  h =  19.43  f =  43.42
It.  55: (  T4,  0.32 L)  g =  23.99  h =  19.43  f =  43.42
It.  56: (  T1,  0.67 L)  g =  23.79  h =  19.90  f =  43.69
It.  57: (  T2,  0.56 L)  g =  23.79  h =  19.90  f =  43.69
It.  58: (  T3,  0.47 L)  g =  23.79  h =  19.90  f =  43.69
It.  59: (  T4,  0.30 L)  g =  23.79  h =  19.90  f =  43.69
It.  60: (CHAP,  0.86 L)  g =  23.89  h =  20.37  f =  44.26
It.  61: (  T1,  0.65 L)  g =  23.89  h =  20.37  f =  44.26
It.  62: (  T2,  0.54 L)  g =  23.89  h =  20.37  f =  44.26
It.  63: (  T3,  0.45 L)  g =  23.89  h =  20.37  f =  44.26
It.  64: (  T4,  0.28 L)  g =  23.89  h =  20.37  f =  44.26
It.  65: (  T1,  0.63 L)  g =  23.89  h =  20.85  f =  44.74
It.  66: (  T2,  0.52 L)  g =  23.89  h =  20.85  f =  44.74
It.  67: (  T3,  0.43 L)  g =  23.89  h =  20.85  f =  44.74
It.  68: (  T4,  0.26 L)  g =  23.89  h =  20.85  f =  44.74
It.  69: (  T3,  0.41 L)  g =  23.79  h =  21.32  f =  45.11
It.  70: (  T4,  0.24 L)  g =  23.79  h =  21.32  f =  45.11
It.  71: (  T3,  0.39 L)  g =  23.79  h =  21.79  f =  45.58
It.  72: (  T2,  0.48 L)  g =  23.79  h =  21.79  f =  45.58
It.  73: (  T4,  0.22 L)  g =  23.79  h =  21.79  f =  45.58
It.  74: (  J1,  0.73 L)  g =  23.79  h =  22.27  f =  46.06
It.  75: (  T2,  0.46 L)  g =  23.79  h =  22.27  f =  46.06
It.  76: (  T3,  0.37 L)  g =  23.79  h =  22.27  f =  46.06
It.  77: (  T4,  0.20 L)  g =  23.79  h =  22.27  f =  46.06
It.  78: (  T3,  0.35 L)  g =  23.79  h =  22.74  f =  46.53
It.  79: (  T1,  0.55 L)  g =  23.79  h =  22.74  f =  46.53
It.  80: (  T4,  0.18 L)  g =  23.79  h =  22.74  f =  46.53
It.  81: (  T2,  0.44 L)  g =  23.79  h =  22.74  f =  46.53
It.  82: (  T1,  0.53 L)  g =  23.79  h =  23.22  f =  47.01
It.  83: (  T2,  0.42 L)  g =  23.79  h =  23.22  f =  47.01
It.  84: (  T3,  0.33 L)  g =  23.79  h =  23.22  f =  47.01
It.  85: (  T4,  0.16 L)  g =  23.79  h =  23.22  f =  47.01
It.  86: (  T4,  1.92 L)  g =  47.48  h =   0.00  f =  47.48
It.  87: (  T4,  1.74 L)  g =  47.48  h =   0.00  f =  47.48
```

```
It.  88: (  T4,  1.58 L)  g =  47.48  h =   0.00  f =  47.48
It.  89: (  T4,  1.56 L)  g =  47.48  h =   0.00  f =  47.48
It.  90: (  T4,  1.52 L)  g =  47.48  h =   0.00  f =  47.48
It.  91: (  T4,  1.40 L)  g =  47.48  h =   0.00  f =  47.48
It.  92: (  T4,  1.38 L)  g =  47.48  h =   0.00  f =  47.48
It.  93: (  T4,  1.34 L)  g =  47.48  h =   0.00  f =  47.48
It.  94: (  T4,  1.30 L)  g =  47.48  h =   0.00  f =  47.48
It.  95: (  T4,  1.24 L)  g =  47.48  h =   0.00  f =  47.48
It.  96: (  T4,  1.22 L)  g =  47.48  h =   0.00  f =  47.48
It.  97: (  T4,  1.20 L)  g =  47.48  h =   0.00  f =  47.48
It.  98: (  T4,  1.18 L)  g =  47.48  h =   0.00  f =  47.48
It.  99: (  T4,  1.16 L)  g =  47.48  h =   0.00  f =  47.48
It. 100: (  T3,  1.75 L)  g =  47.48  h =   0.00  f =  47.48
It. 101: (SUCH,  0.99 L)  g =  47.48  h =   0.00  f =  47.48
It. 102: (  T3,  1.57 L)  g =  47.48  h =   0.00  f =  47.48
It. 103: (SUCH,  0.81 L)  g =  47.48  h =   0.00  f =  47.48
It. 104: (  T3,  1.41 L)  g =  47.48  h =   0.00  f =  47.48
It. 105: (SUCH,  0.65 L)  g =  47.48  h =   0.00  f =  47.48
It. 106: (  T3,  1.39 L)  g =  47.48  h =   0.00  f =  47.48
It. 107: (SUCH,  0.63 L)  g =  47.48  h =   0.00  f =  47.48
It. 108: (  T3,  1.35 L)  g =  47.48  h =   0.00  f =  47.48
It. 109: (SUCH,  0.59 L)  g =  47.48  h =   0.00  f =  47.48
It. 110: (SUCH,  0.47 L)  g =  47.48  h =   0.00  f =  47.48
It. 111: (SUCH,  0.45 L)  g =  47.48  h =   0.00  f =  47.48
It. 112: (SUCH,  0.41 L)  g =  47.48  h =   0.00  f =  47.48
It. 113: (SUCH,  0.37 L)  g =  47.48  h =   0.00  f =  47.48
It. 114: (SUCH,  0.31 L)  g =  47.48  h =   0.00  f =  47.48
It. 115: (SUCH,  0.29 L)  g =  47.48  h =   0.00  f =  47.48
It. 116: (SUCH,  0.27 L)  g =  47.48  h =   0.00  f =  47.48
It. 117: (SUCH,  0.25 L)  g =  47.48  h =   0.00  f =  47.48
It. 118: (SUCH,  0.23 L)  g =  47.48  h =   0.00  f =  47.48
It. 119: (  T2,  1.66 L)  g =  47.48  h =   0.00  f =  47.48
It. 120: (  T4,  1.58 L)  g =  47.48  h =   0.00  f =  47.48
It. 121: (  UP,  0.78 L)  g =  47.48  h =   0.00  f =  47.48
```

Listing 5.1: Iteraciones de A* en el Caso 1.

5.4. Anexo D: Repositorio del proyecto

El código fuente completo, este reporte y las imágenes se encuentran en el repositorio del proyecto en GitHub, con la estructura descrita en la Tabla 2.1. Las imágenes del grafo se elaboraron con Graphviz y el diagrama de flujo con Mermaid.

Bibliografía

[1] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[2] Google. Google maps y maps urls, 2026. Accesado el 2026.11.01. Disponible en `https://developers.google.com/maps/documentation/urls/get-started`.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[4] Carlos Alberto Díaz Hernández. Fundamentos de inteligencia artificial: agentes y búsqueda no informada e informada, 2026. Diapositivas del curso. Universidad Politécnica de Chiapas.

[5] Python Software Foundation. dataclasses – data classes, 2026. Accesado el 2026.11.01. Disponible en `https://docs.python.org/3/library/dataclasses.html`.

[6] Python Software Foundation. heapq – heap queue algorithm, 2026. Accesado el 2026.11.01. Disponible en `https://docs.python.org/3/library/heapq.html`.

[7] Python Software Foundation. tkinter – python interface to tcl/tk, 2026. Accesado el 2026.11.01. Disponible en `https://docs.python.org/3/library/tkinter.html`.

[8] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

[9] University of Toronto. CSC384: Introduction to artificial intelligence – heuristic search, 2020. Diapositivas del curso.